

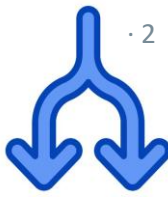
Practical Concurrent and Parallel Programming VII

Linearizability

Raúl Pardo

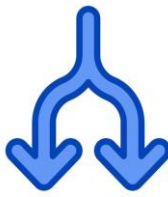
Based on slides of PCPP 2019

Change in exercise rooms for today



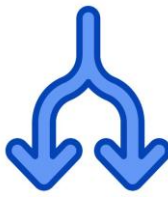
- The exercise session today will be conducted in rooms 3A52 and 3A54

Previously on PCPP...



- Compare-And-Swap (CAS)
 - Lock-free atomic integer
 - Lock-free number range
 - Atomic libraries
 - CAS based lock implementation
- Lock-free stack
- ABA problem
- Lock-free queue

Agenda



- Revisit
 - Progress conditions
 - Lock-free queue
- Linearizability
 - Sequential consistency
 - Definition of Linearizability
 - Linearizable concurrent objects
- Work-stealing queues

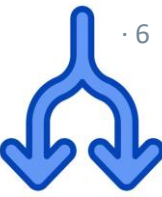
Progress conditions for non-blocking DS



There exist 3 main notions of progress in non-blocking data structures/computation

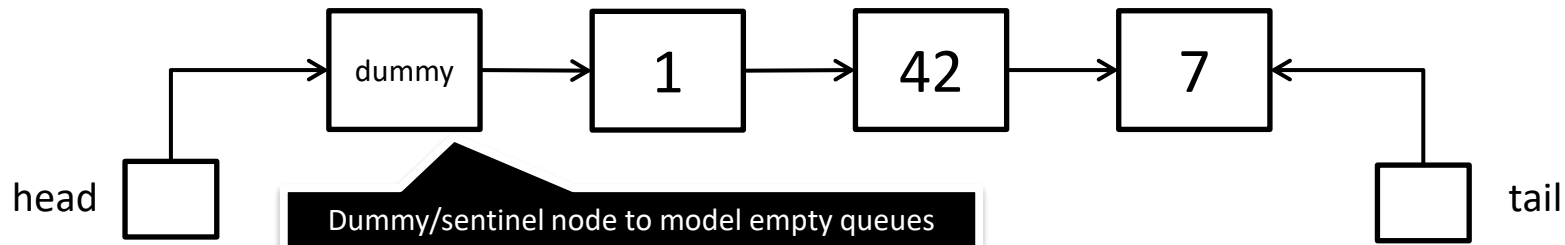
1. Wait-free: A method of an object implementation is wait-free if every call finishes its execution in a finite number of steps
 - My operations are guaranteed to complete in a bounded number of steps (no matter what other threads do)
2. Lock-free: A method of an object implementation is lock-free if executing the method guarantees that some method call (including concurrent) finishes in a finite number of steps
 - Somebody's operations are guaranteed to complete in a bounded number of steps
 - Most non-blocking data structures are lock-free, e.g., Treap's stack
3. Obstruction-free: A method of an object implementation is obstruction-free if, from any point after which it executes in isolation, it finishes in a finite number of steps;
 - My operations are guaranteed to complete in a bounded number of steps (if I get to execute them)

$$wait_{free} \Rightarrow lock_{free} \Rightarrow obstruction_{free}$$



Lock-free data structures: Queue

- A queue is a data structure following a FIFO (*first-in-first-out*) policy
 - enqueue() – adds an element to the tail of the queue
 - dequeue() – removes an element from the head of the queue
- It is typically implemented as a linked list



Lock-free queue

- Michael-Scott lock free queue, introduced in 1996 (see optional readings)
- Implemented in ConcurrentLinkedQueue in java.concurrent.* by Doug Lea et. al. (see [here](#))
 - The version on the right is not the JDK implementation

```
private static class Node<T> {  
    final T item;  
    final AtomicReference<Node<T>> next;  
  
    public Node(T item, Node<T> next) {  
        this.item = item;  
        this.next = new AtomicReference<Node<T>>(next);  
    }  
}
```

```
class MSQueue<T> implements UnboundedQueue<T> {  
    private final AtomicReference<Node<T>> head, tail;  
  
    public MSQueue() {  
        Node<T> dummy = new Node<T>(null, null);  
        head = new AtomicReference<Node<T>>(dummy);  
        tail = new AtomicReference<Node<T>>(dummy);  
    }  
  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get(), next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
            }  
        }  
    }  
  
    public T dequeue() {  
        while (true) {  
            Node<T> first = head.get(), last = tail.get(), next = first.next.get();  
            if (first == head.get()) {  
                if (first == last) {  
                    if (next == null)  
                        return null;  
                    else  
                        tail.compareAndSet(last, next);  
                } else {  
                    T result = next.item;  
                    if (head.compareAndSet(first, next))  
                        return result;  
                }  
            }  
        }  
    }  
}
```

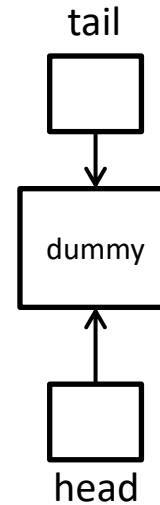
See MSQueue.java (in lecture 6 code)



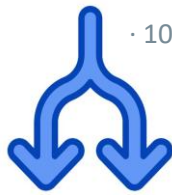
Lock-free queue | initialization



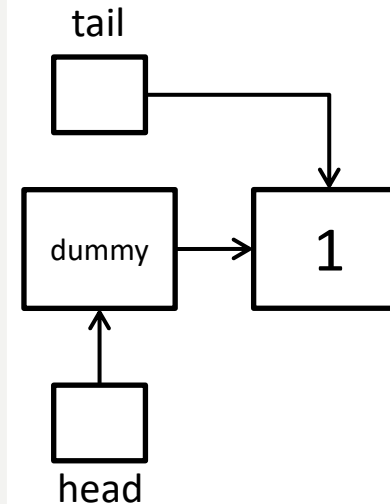
```
class MSQueue<T> implements UnboundedQueue<T> {  
    private final AtomicReference<Node<T>> head, tail;  
  
    public MSQueue() {  
        Node<T> dummy = new Node<T>(null, null);  
        head = new AtomicReference<Node<T>>(dummy);  
        tail = new AtomicReference<Node<T>>(dummy);  
    }  
    ...  
}
```



Lock-free queue | enqueue

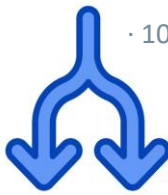


```
class MSQueue<T> implements UnboundedQueue<T> {
...
public void enqueue(T item) {
    Node<T> node = new Node<T>(item, null);
    while (true) {
        Node<T> last = tail.get();
        Node<T> next = last.next.get();
        if (last == tail.get()) {
            if (next == null) {
                // In quiescent state, try inserting new node
                if (last.next.compareAndSet(next, node)) {
                    // Insertion succeeded, try advancing tail
                    tail.compareAndSet(last, node);
                    return;
                }
            } else {
                // Queue in intermediate state, advance tail
                tail.compareAndSet(last, next);
            }
        }
    }
}
...
}
```

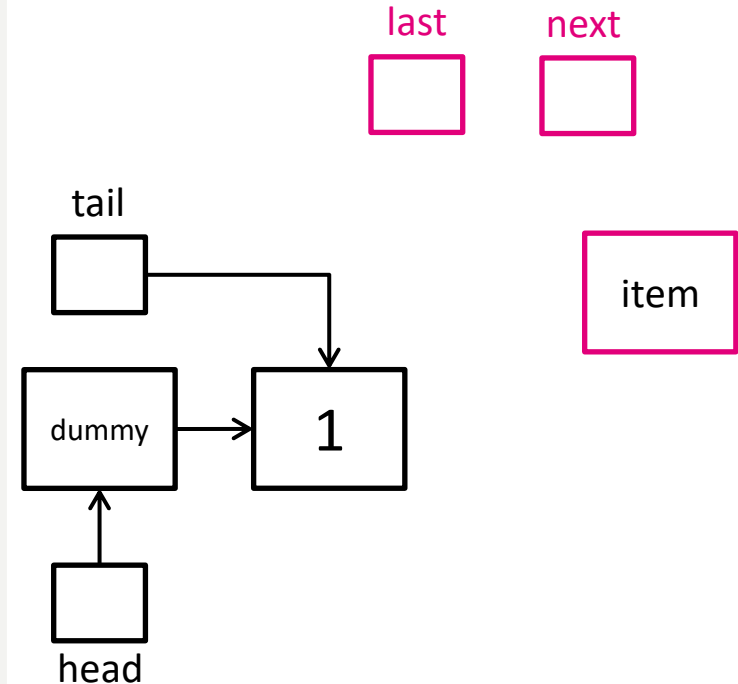


Lock-free queue | enqueue

· 10



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
            }  
        }  
    }  
...  
}
```

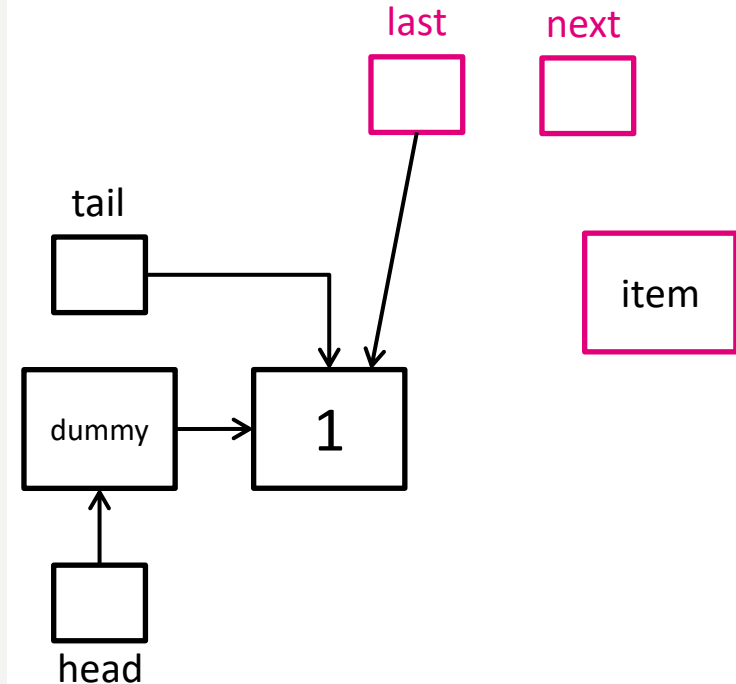


Lock-free queue | enqueue

· 10



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
            }  
        }  
    }  
...  
}
```

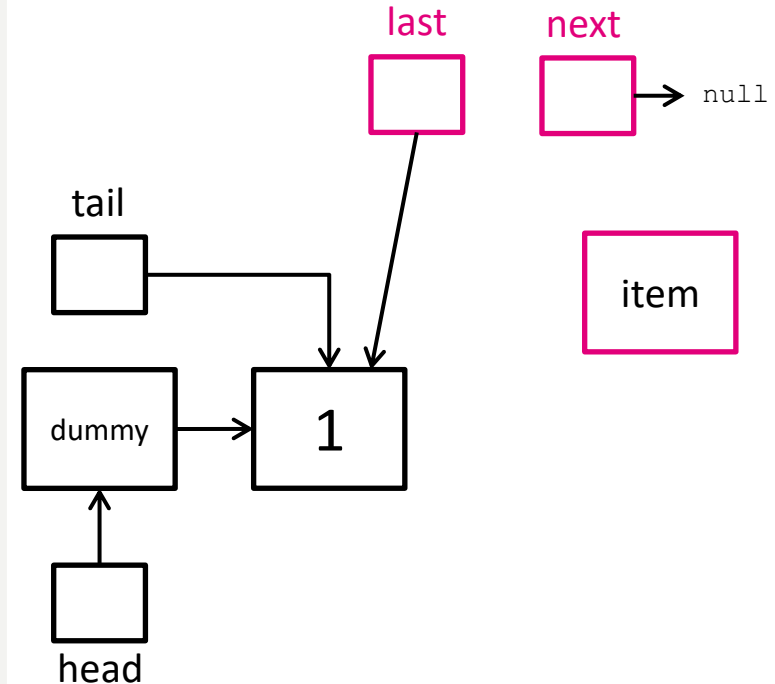


Lock-free queue | enqueue

· 10



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
            }  
        }  
    }  
...  
}
```

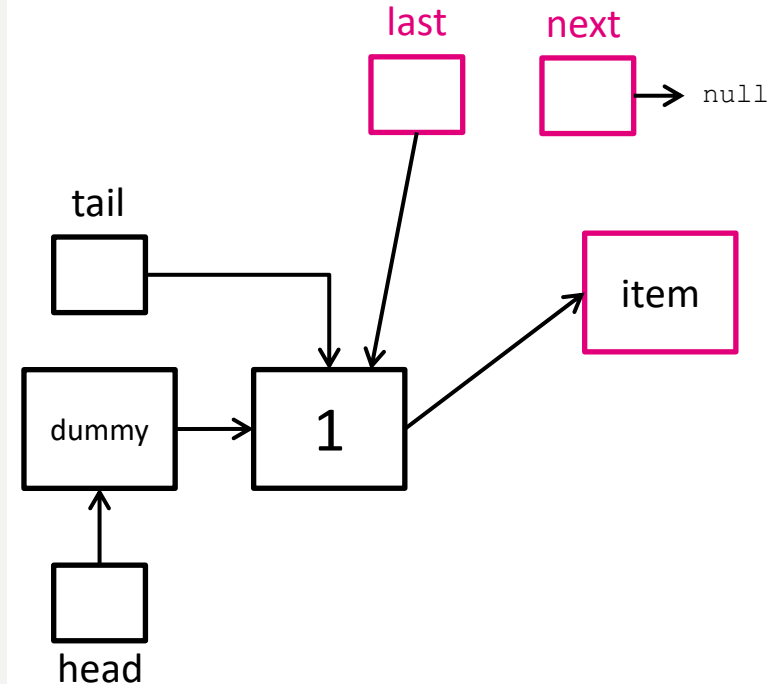


Lock-free queue | enqueue

· 10



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
            }  
        }  
    }  
...  
}
```

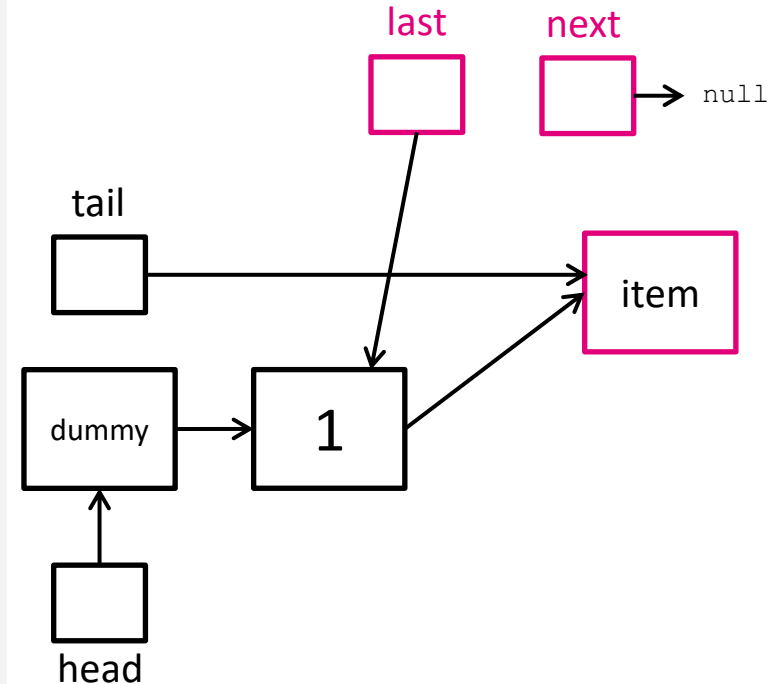


Lock-free queue | enqueue

· 10



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
            }  
        }  
    }  
}
```



Lock-free queue | enqueue

· 11

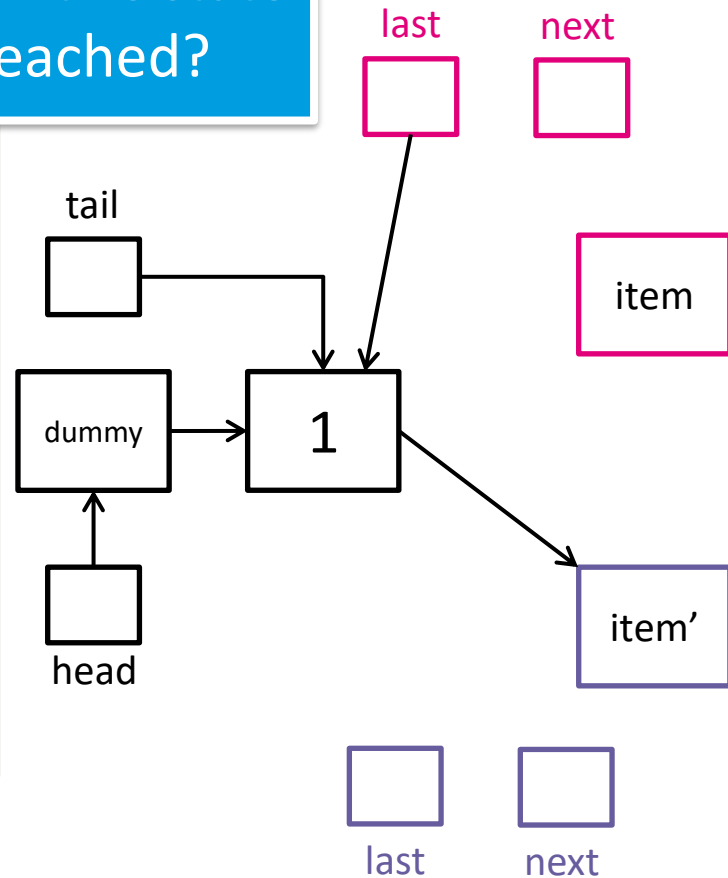


How can this state be reached?

```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
public void enqueue(T item) {  
    Node<T> node = new Node<T>(item, null);  
    while (true) {  
        Node<T> last = tail.get();  
        Node<T> next = last.next.get();  
        if (last == tail.get()) {  
            if (next == null) {  
                // In quiescent state, try inserting new node  
                if (last.next.compareAndSet(next, node)) {  
                    // Insertion succeeded, try advancing tail  
                    tail.compareAndSet(last, node);  
                    return;  
                }  
            } else  
                // Queue in intermediate state, advance tail  
                tail.compareAndSet(last, next);  
        }  
    }  
}
```



menti.com
6658 7456

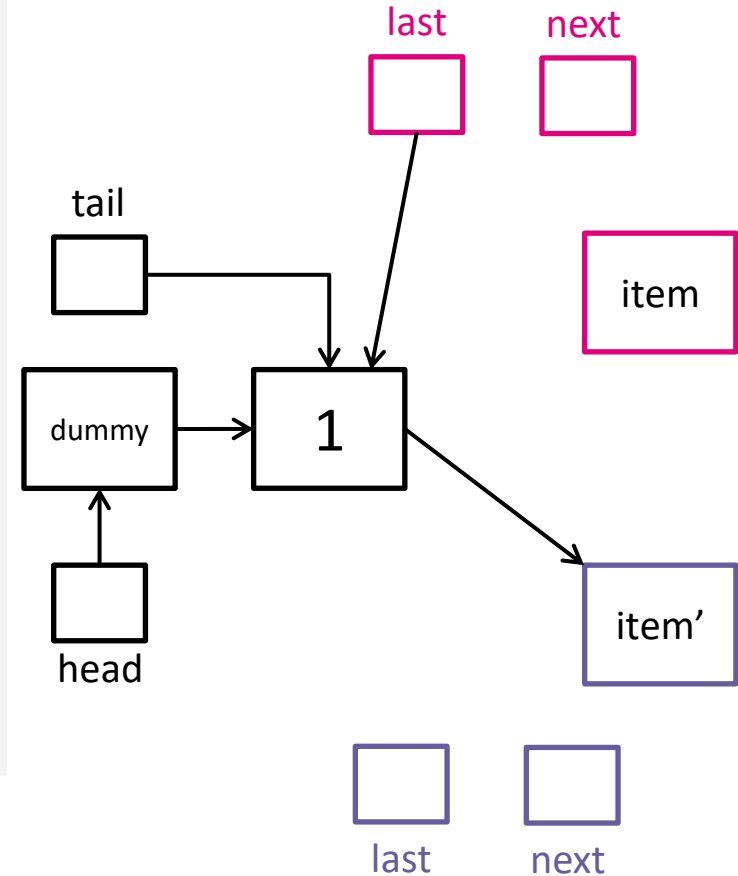


Lock-free queue | enqueue

· 11



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else {  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
                }  
            }  
        }  
    }  
...  
}
```

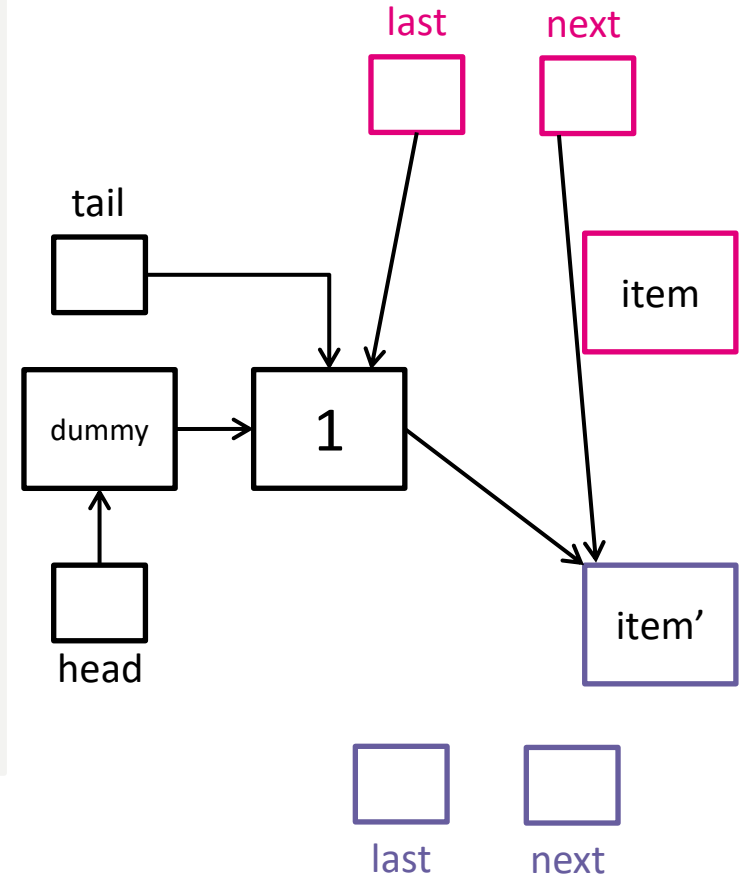


Lock-free queue | enqueue

· 11



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else {  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
                }  
            }  
        }  
    }  
}
```

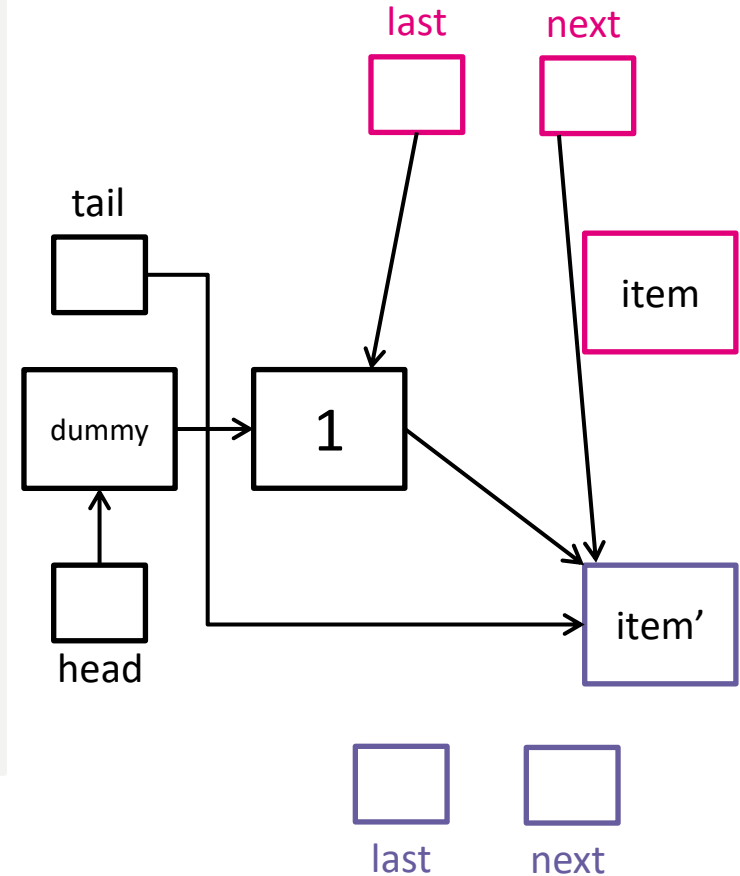


Lock-free queue | enqueue

· 11



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else {  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
                }  
            }  
        }  
    }  
}
```



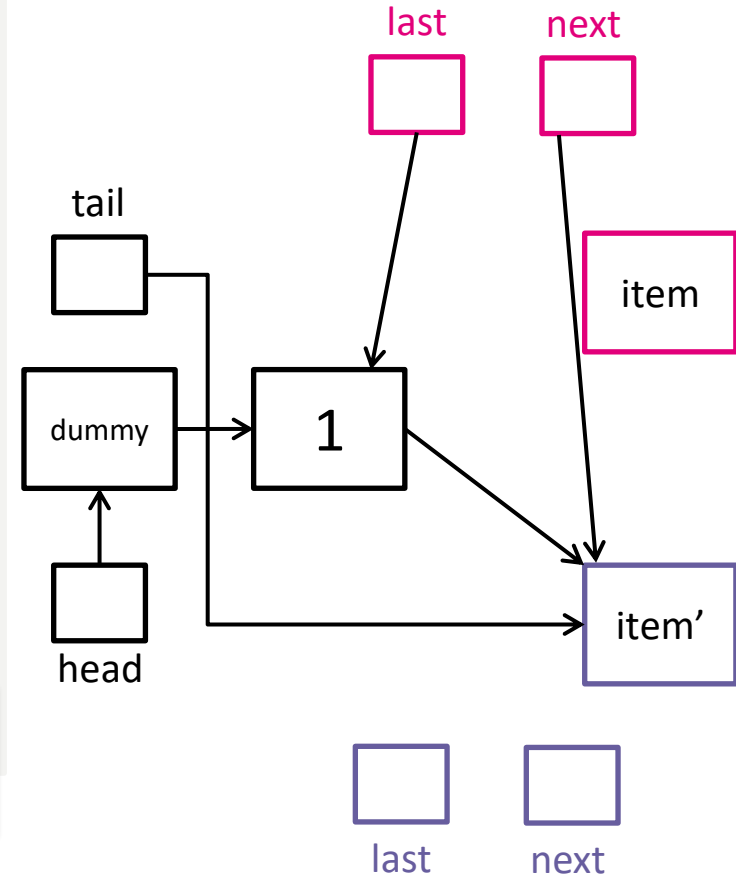
Lock-free queue | enqueue

· 11



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else {  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
                }  
            }  
        }  
    }  
}
```

In case another thread is enqueueing, and didn't update the tail, the current thread helps by advancing the tail



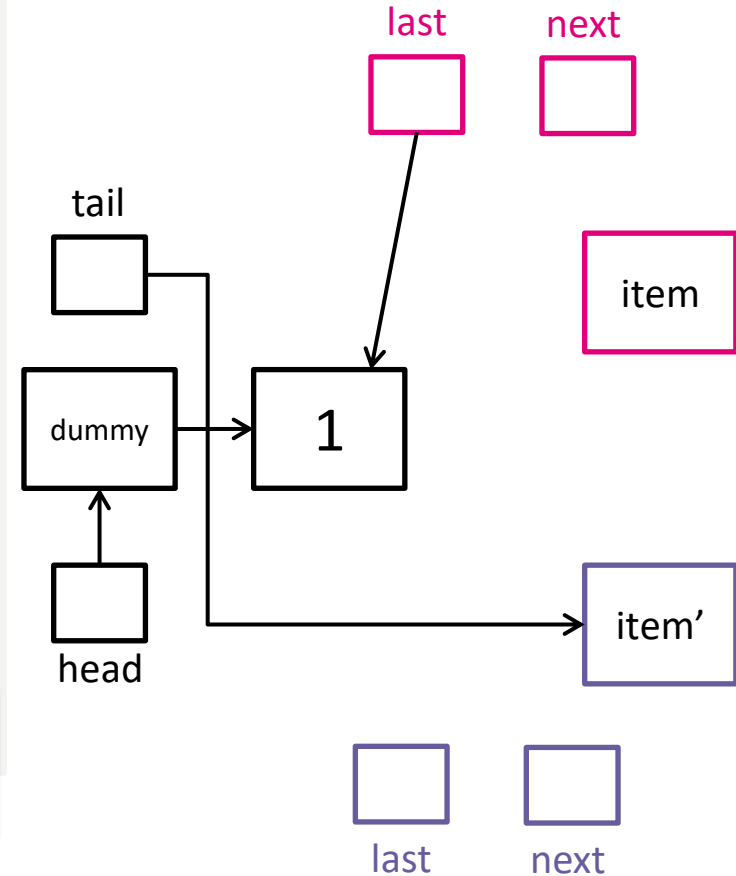
Lock-free queue | enqueue

· 12



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) {  
                if (next == null) {  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) {  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
            }  
        }  
    }  
}
```

If the tail has been changed, then the thread restarts right away

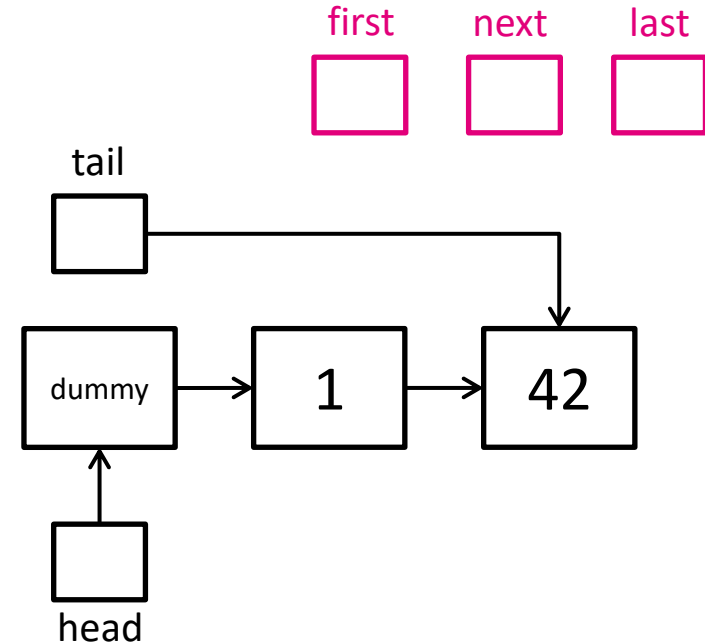


Lock-free queue | dequeue

· 13



```
class MSQueue<T> implements UnboundedQueue<T> {  
    ...  
  
    public T dequeue() {  
        while (true) {  
            Node<T> first = head.get();  
            Node<T> last = tail.get();  
            Node<T> next = first.next.get();  
            if (first == head.get()) {  
                if (first == last) {  
                    if (next == null)  
                        return null;  
                    else  
                        tail.compareAndSet(last, next);  
                } else {  
                    T result = next.item;  
                    if (head.compareAndSet(first, next))  
                        return result;  
                }  
            }  
        }  
    }  
    ...  
}
```

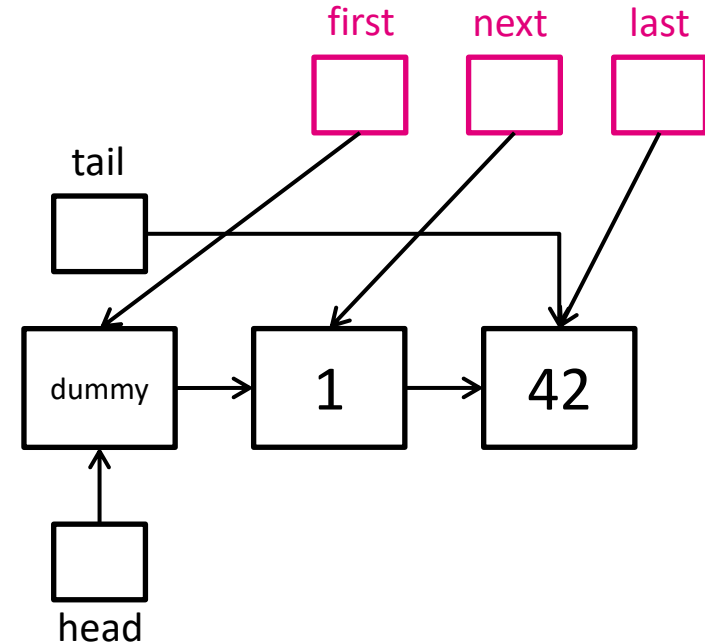


Lock-free queue | dequeue

· 13



```
class MSQueue<T> implements UnboundedQueue<T> {  
    ...  
  
    public T dequeue() {  
        while (true) {  
            Node<T> first = head.get();  
            Node<T> last = tail.get();  
            Node<T> next = first.next.get();  
            if (first == head.get()) {  
                if (first == last) {  
                    if (next == null)  
                        return null;  
                    else  
                        tail.compareAndSet(last, next);  
                } else {  
                    T result = next.item;  
                    if (head.compareAndSet(first, next))  
                        return result;  
                }  
            }  
        }  
    }  
    ...  
}
```

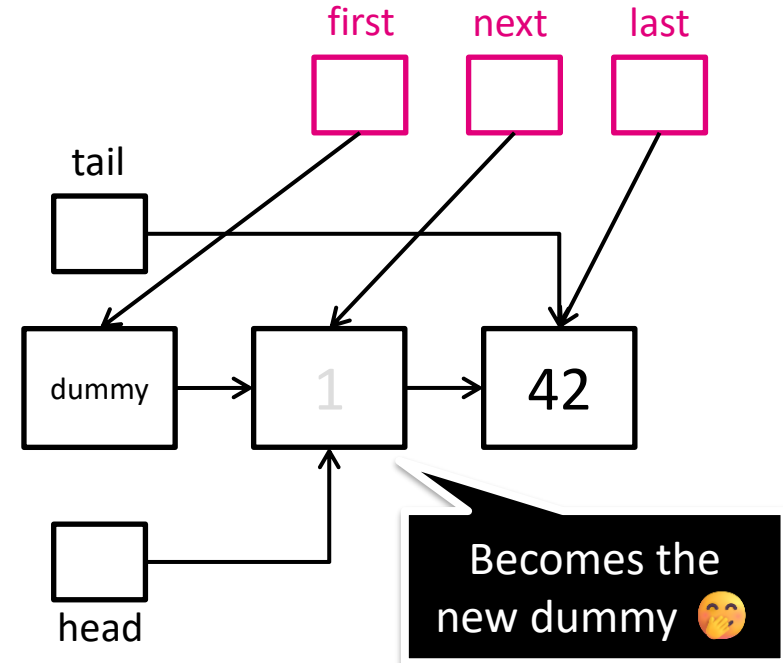


Lock-free queue | dequeue

· 13



```
class MSQueue<T> implements UnboundedQueue<T> {  
    ...  
  
    public T dequeue() {  
        while (true) {  
            Node<T> first = head.get();  
            Node<T> last = tail.get();  
            Node<T> next = first.next.get();  
            if (first == head.get()) {  
                if (first == last) {  
                    if (next == null)  
                        return null;  
                    else  
                        tail.compareAndSet(last, next);  
                } else {  
                    T result = next.item;  
                    if (head.compareAndSet(first, next))  
                        return result;  
                }  
            }  
        }  
    }  
    ...  
}
```

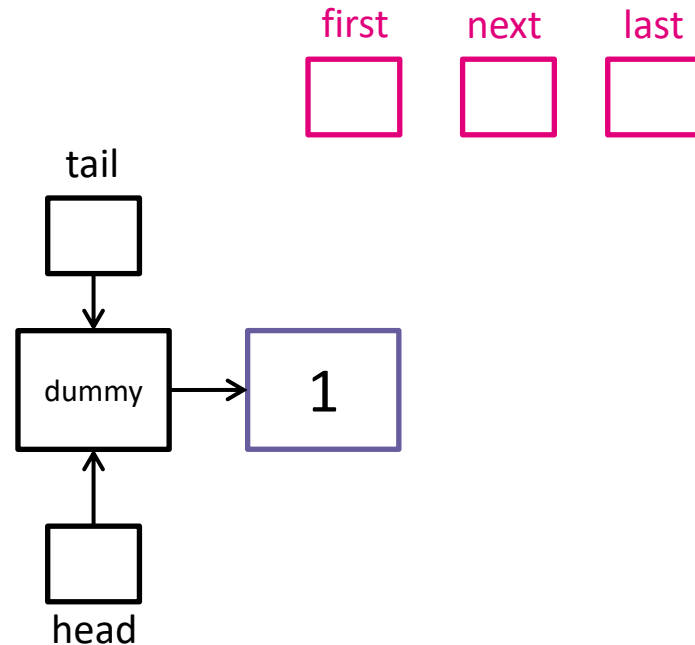


Lock-free queue | dequeue

· 14



```
class MSQueue<T> implements UnboundedQueue<T> {  
    ...  
  
    public T dequeue() {  
        while (true) {  
            Node<T> first = head.get();  
            Node<T> last = tail.get();  
            Node<T> next = first.next.get();  
            if (first == head.get()) {  
                if (first == last) {  
                    if (next == null)  
                        return null;  
                    else  
                        tail.compareAndSet(last, next);  
                } else {  
                    T result = next.item;  
                    if (head.compareAndSet(first, next))  
                        return result;  
                }  
            }  
        }  
    }  
    ...  
}
```



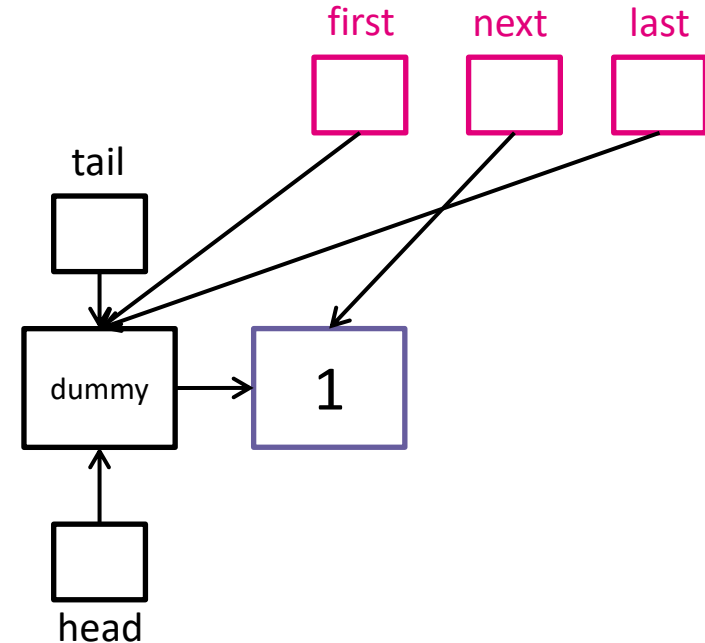
Lock-free queue | dequeue

· 14



```
class MSQueue<T> implements UnboundedQueue<T> {
...

    public T dequeue() {
        while (true) {
            Node<T> first = head.get();
            Node<T> last = tail.get();
            Node<T> next = first.next.get();
            if (first == head.get()) {
                if (first == last) {
                    if (next == null)
                        return null;
                    else
                        tail.compareAndSet(last, next);
                } else {
                    T result = next.item;
                    if (head.compareAndSet(first, next))
                        return result;
                }
            }
        }
    }
...
}
```



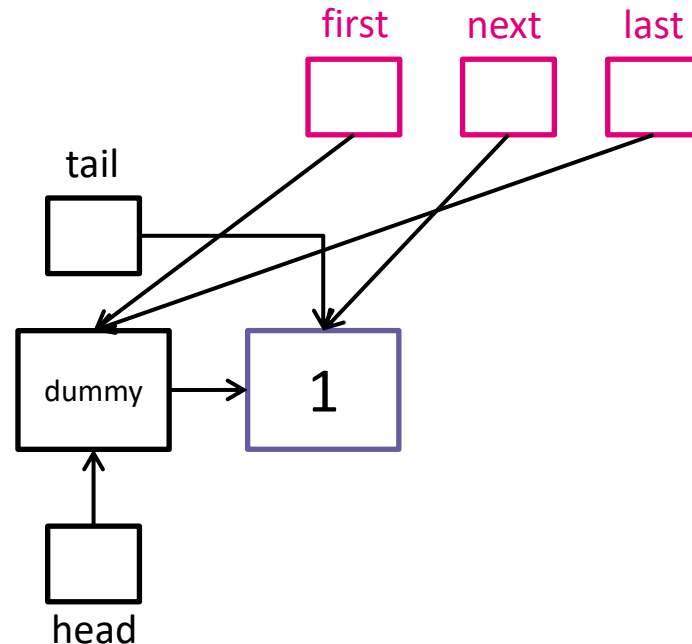
Lock-free queue | dequeue

· 14



```
class MSQueue<T> implements UnboundedQueue<T> {  
    ...  
  
    public T dequeue() {  
        while (true) {  
            Node<T> first = head.get();  
            Node<T> last = tail.get();  
            Node<T> next = first.next.get();  
            if (first == head.get()) {  
                if (first == last) {  
                    if (next == null)  
                        return null;  
                    else  
                        tail.compareAndSet(last, next);  
                } else {  
                    T result = next.item;  
                    if (head.compareAndSet(first, next))  
                        return result;  
                }  
            }  
        }  
    }  
}
```

If the next field of the head is not null (because another thread enqueued in the meantime), then the calling thread helps advancing the tail and tries to pop again.

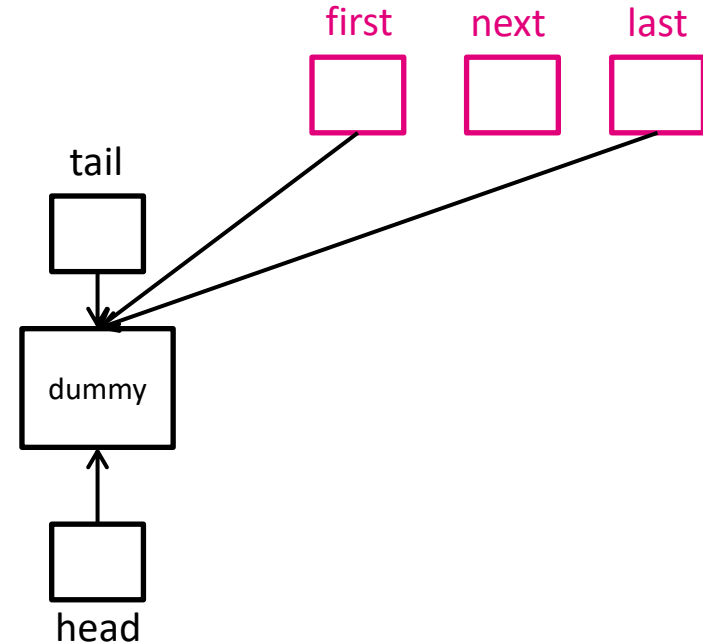


Lock-free queue | dequeue

· 15



```
class MSQueue<T> implements UnboundedQueue<T> {  
    ...  
  
    public T dequeue() {  
        while (true) {  
            Node<T> first = head.get();  
            Node<T> last = tail.get();  
            Node<T> next = first.next.get();  
            if (first == head.get()) {  
                if (first == last) {  
                    if (next == null)  
                        return null;  
                    else  
                        tail.compareAndSet(last, next);  
                } else {  
                    T result = next.item;  
                    if (head.compareAndSet(first, next))  
                        return result;  
                }  
            }  
        }  
    }  
    ...  
}
```

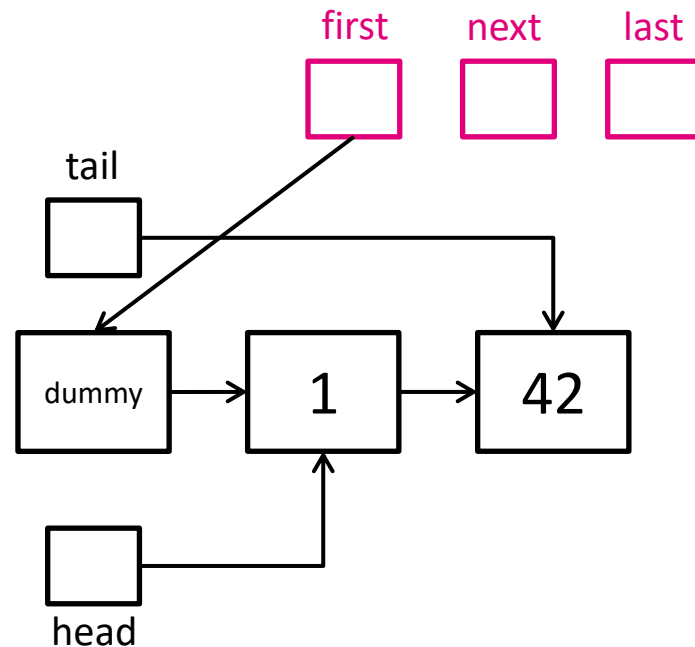


Lock-free queue | dequeue

· 16



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
  
    public T dequeue() {  
        while (true) {  
            Node<T> first = head.get();  
            Node<T> last = tail.get();  
            Node<T> next = first.next.get();  
            if (first == head.get()) {  
                if (first == last) {  
                    if (next == null)  
                        return null;  
                    else  
                        tail.compareAndSet(last, next);  
                } else {  
                    T result = next.item;  
                    if (head.compareAndSet(first, next))  
                        return result;  
                }  
            }  
        }  
    }  
...  
}
```



What about correctness?



- We have seen several implementations of lock-free data structures today and last week
- However, we have not seen any techniques to reason about their correctness...
 - How do we even write the *concurrent* specification of the lock-free queue we have seen?

Linearizability



- Defining correctness of (non-blocking) concurrent objects is tricky
 - Recall correctness is defined by a specification
- ***The motivation behind linearizability is to use specifications of sequential objects as a basis for the correctness of concurrent objects***
 - Specifications of sequential objects are typically expressed as pre- and post-conditions for method calls



```
class WaitFreeQueue<T> {
    int head = 0, tail = 0;
    T[] items;

    public WaitFreeQueue(int capacity) {
        items = (T[]) new Object[capacity]
    }

    public void enq(T x) throws Exception {
        if (tail - head == items.length)
            throw new Exception();
        items[tail % items.length] = x;
        tail++;
    }

    public void deq() throws Exception {
        if (tail - head == 0)
            throw new Exception();
        T x = items[head % items.length];
        head++;
        return x;
    }
}
```

- Informally, we would like concurrent operations to happen as if they were executed sequentially
- The specification of a concurrent queue is the sequential behaviour of queues

Wait Free Queue

· 21



```
class WaitFreeQueue<T> {
    int head = 0, tail = 0;
    T[] items;

    public WaitFreeQueue(int capacity) {
        items = (T[]) new Object[capacity]
    }

    public void enq(T x) throws Exception {
        if (tail - head == items.length)
            throw new Exception();
        items[tail % items.length] = x;
        tail++;
    }

    public void deq() throws Exception {
        if (tail - head == 0)
            throw new Exception();
        T x = items[head % items.length];
        head++;
        return x;
    }
}
```

This is pseudo-code for illustration purposes.
Assume all variables have volatile semantics
and each program line is atomic.

Are there any problems with
this implementation?



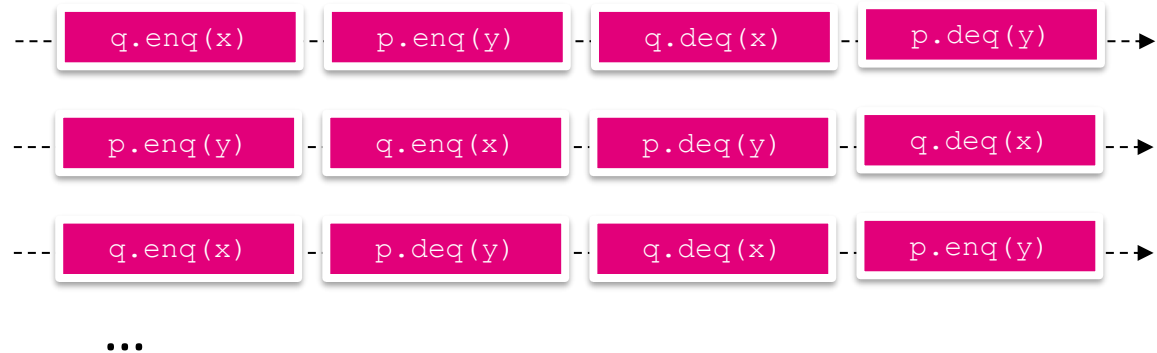
menti.com
6658 7456

Sequential consistency



- Recall that, *in sequential programs*, the compiler and CPU are allowed to re-order instructions as long as the re-ordering does not change the result of the program
- When executed sequentially, (some of) these are possible orderings for execution:

```
q.enq(x)
q.deq(x)
p.enq(y)
p.deq(y)
```

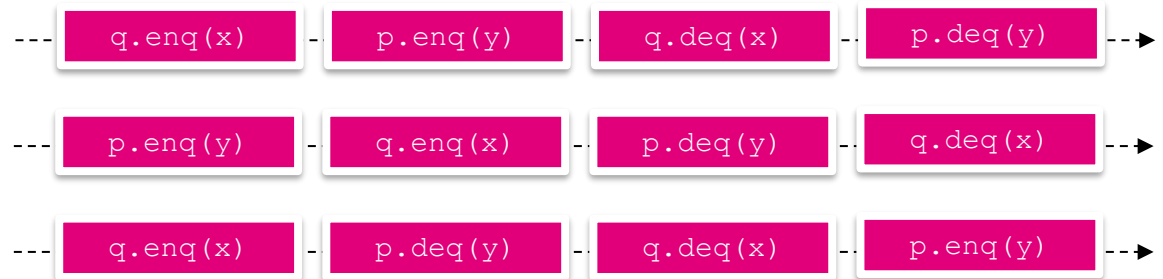


Sequential consistency



- Recall that, *in sequential programs*, the compiler and CPU are allowed to re-order instructions as long as the re-ordering does not change the result of the program
- When executed sequentially, (some of) these are possible orderings for execution:

```
q.enq(x)
q.deq(x)
p.enq(y)
p.deq(y)
```



This syntax means: The pink thread enqueues x on queue q successfully

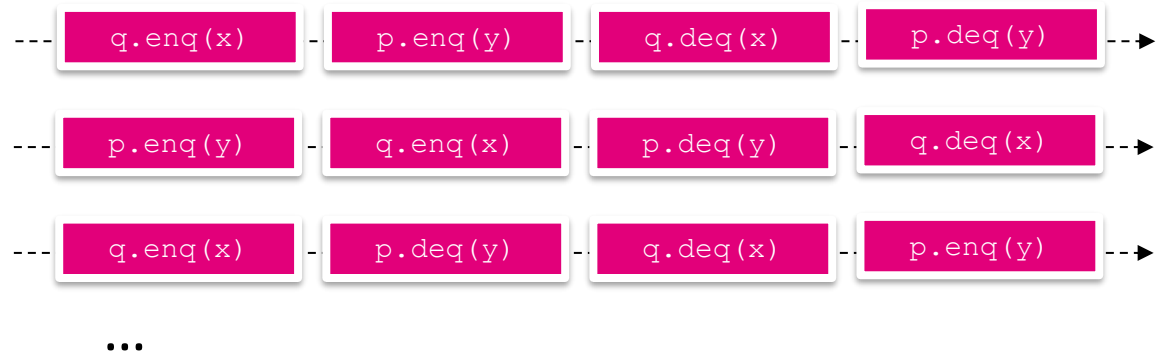
This syntax means: The pink thread dequeues x from queue q successfully

Sequential consistency



- Recall that, *in sequential programs*, the compiler and CPU are allowed to re-order instructions as long as the re-ordering does not change the result of the program
- When executed sequentially, (some of) these are possible orderings for execution:

```
q.enq(x)
q.deq(x)
p.enq(y)
p.deq(y)
```

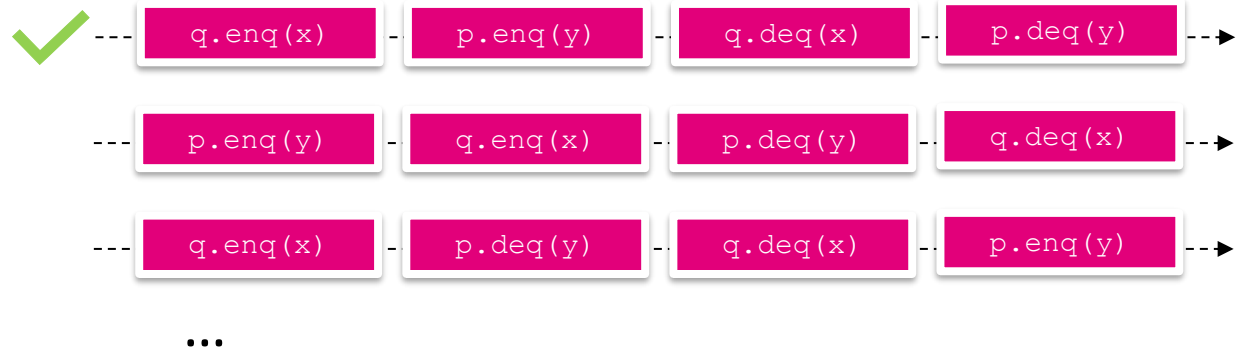


Sequential consistency



- Recall that, *in sequential programs*, the compiler and CPU are allowed to re-order instructions as long as the re-ordering does not change the result of the program
 - When executed sequentially, (some of) these are possible orderings for execution:

```
q.enq(x)
q.deq(x)
p.enq(y)
p.deq(y)
```

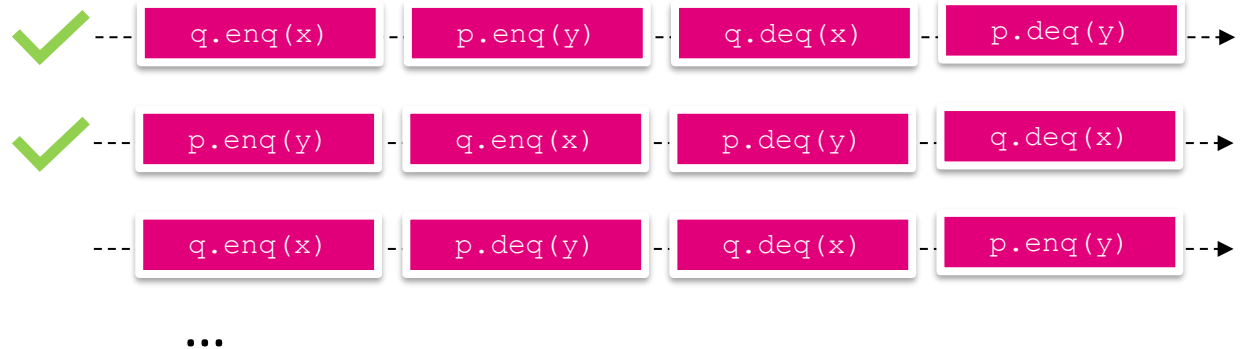


Sequential consistency



- Recall that, *in sequential programs*, the compiler and CPU are allowed to re-order instructions as long as the re-ordering does not change the result of the program
 - When executed sequentially, (some of) these are possible orderings for execution:

```
q.enq(x)
q.deq(x)
p.enq(y)
p.deq(y)
```

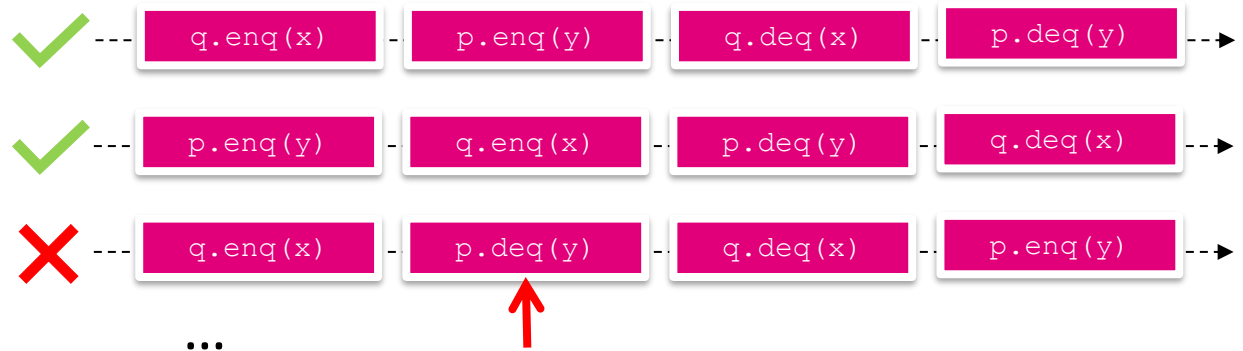


Sequential consistency



- Recall that, *in sequential programs*, the compiler and CPU are allowed to re-order instructions as long as the re-ordering does not change the result of the program
 - When executed sequentially, (some of) these are possible orderings for execution:

```
q.enq(x)
q.deq(x)
p.enq(y)
p.deq(y)
```



- For concurrent executions, we must define the conditions asserting that every thread is behaving consistently w.r.t. a sequential execution
- For executions of concurrent objects, an execution is sequentially consistent iff
 1. Method calls appear to happen in a one-at-a-time, sequential order
 2. Method calls should appear to take effect in program order

- For concurrent executions, we must define the conditions asserting that every thread is behaving consistently w.r.t. a sequential execution
- For executions of concurrent objects, an execution is sequentially consistent iff
 1. Method calls appear to happen in a one-at-a-time, sequential order

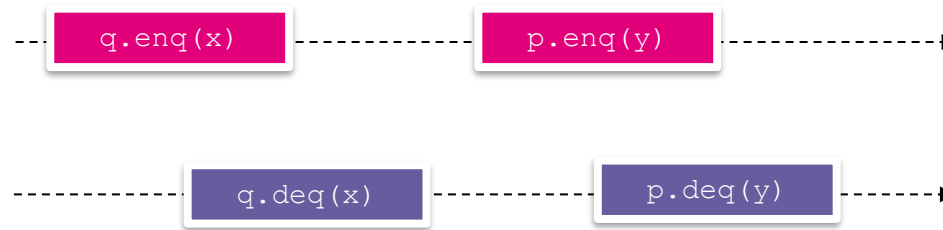
Requires memory commands to be executed in order
 2. Method calls should appear to take effect in program order

- For concurrent executions, we must define the conditions asserting that every thread is behaving consistently w.r.t. a sequential execution
- For executions of concurrent objects, an execution is sequentially consistent iff
 1. Method calls appear to happen in a one-at-a-time, sequential order

Requires memory commands to be executed in order
 2. Method calls should appear to take effect in program order

Requires sequential program order for each thread

Sequential consistency



A concurrent execution is sequentially consistent iff there exists a sequential projection of the concurrent execution where:

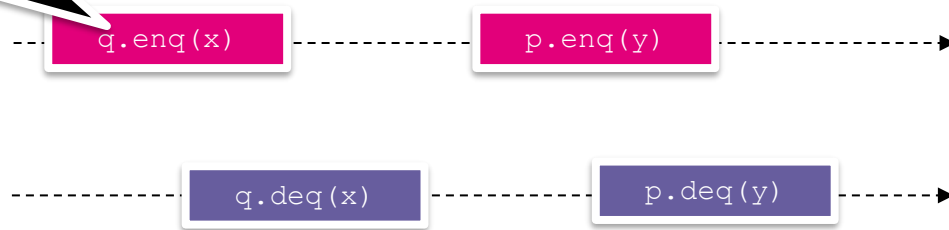
1. Operations happen one-at-a-time
2. Program order is preserved (for each thread)
3. The execution satisfies the specification of the object

Sequential consistency

· 32



This syntax means: The pink thread enqueues x on queue q successfully



A concurrent execution is sequentially consistent iff there exists a sequential projection of the concurrent execution where:

1. Operations happen one-at-a-time
2. Program order is preserved (for each thread)
3. The execution satisfies the specification of the object

Sequential consistency

· 33



This syntax means: The pink thread enqueues x on queue q successfully

$q.\text{enq}(x)$

$p.\text{enq}(y)$

This syntax means: The purple thread dequeues y from queue p successfully

$q.\text{deq}(x)$

$p.\text{deq}(y)$

A concurrent execution is sequentially consistent iff there exists a sequential projection of the concurrent execution where:

1. Operations happen one-at-a-time
2. Program order is preserved (for each thread)
3. The execution satisfies the specification of the object

Sequential consistency

· 34



This syntax means: The pink thread enqueues x on queue q successfully

$q.enq(x)$

The dashed-line represents time. The size of the box represents the time the method call takes; from invocation to response. The position of the box on the line represents the real time moment when the method call takes place

$p.enq(y)$

This syntax means: The purple thread dequeues y from queue p successfully

$q.deq(x)$

$p.deq(y)$

A concurrent execution is sequentially consistent iff there exists a sequential projection of the concurrent execution where:

1. Operations happen one-at-a-time
2. Program order is preserved (for each thread)
3. The execution satisfies the specification of the object

Sequential consistency

· 35



This syntax means: The pink thread enqueues x on queue q successfully

$q.\text{enq}(x)$

The dashed-line represents time. The size of the box represents the time the method call takes; from invocation to response. The position of the box on the line represents the real time moment when the method call takes place

$p.\text{enq}(y)$

This syntax means: The purple thread dequeues y from queue p successfully

$q.\text{deq}(x)$

$p.\text{deq}(y)$

A concurrent execution is sequentially consistent iff there exists a sequential projection of the concurrent execution where:

1. Operations happen one-at-a-time
2. Program order is preserved (for each thread)
3. The execution satisfies the specification of the object



Sequential consistency

· 36



This syntax means: The pink thread enqueues x on queue q successfully

$q.enq(x)$

$p.enq(y)$

The dashed-line represents time. The size of the box represents the time the method call takes; from invocation to response. The position of the box on the line represents the real time moment when the method call takes place

This syntax means: The purple thread dequeues y from queue p successfully

$q.deq(x)$

$p.deq(y)$

A concurrent execution is sequentially consistent iff there exists a sequential projection of the concurrent execution where:

1. Operations happen one-at-a-time

2. Program order is preserved (for each thread)

3. The execution satisfies the specification of the object

Sequential projection of the execution above obtained by “sliding” the operations of each thread; with the only constraint that, within a thread, program order must be preserved

$q.enq(x)$

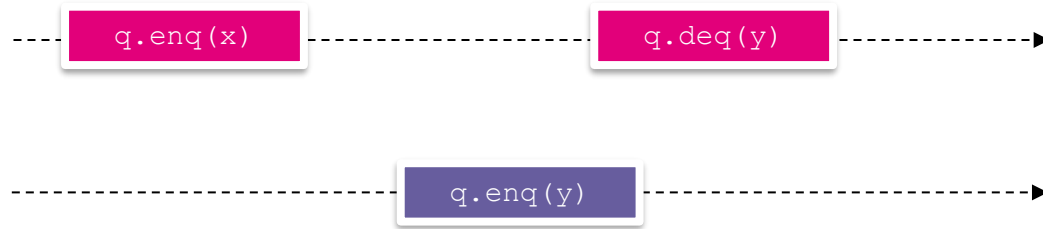
$p.enq(y)$

$q.deq(x)$

$p.deq(y)$

These are instantaneous executions, and the line only represents execution order (not real time)

Sequential consistency



Is this concurrent execution sequentially consistent?
(Recall we are working with a FIFO queue)



menti.com
6658 7456

Sequentially Consistent Concurrent Objects

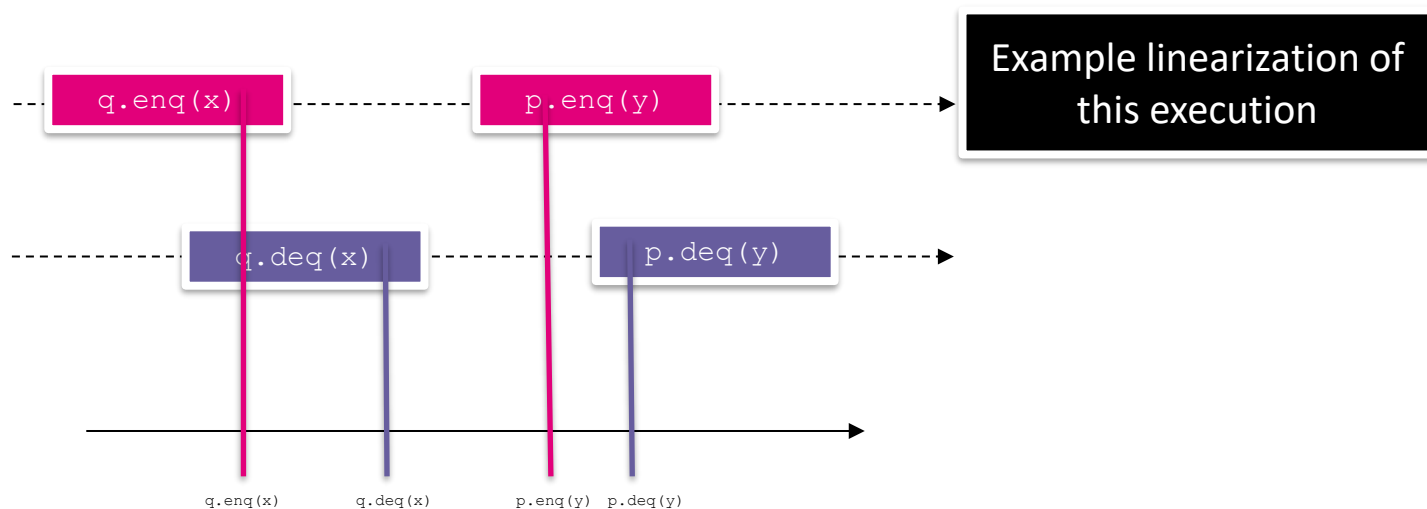


- A concurrent object is sequentially consistent iff all concurrent executions of method calls are sequentially consistent
- Unfortunately, sequential consistency is not a compositional property
 - Concurrent executions involving sequentially consistent objects may not be sequentially consistent
 - This is a major weakness, as it does not allow for modular reasoning when implementing concurrent objects

- Linearizability extends sequential consistency by requiring that the real time order of the execution is preserved
- Linearizability extends sequential consistency with the following condition:
 1. Each method call should appear to take effect instantaneously at some moment between its invocation and response

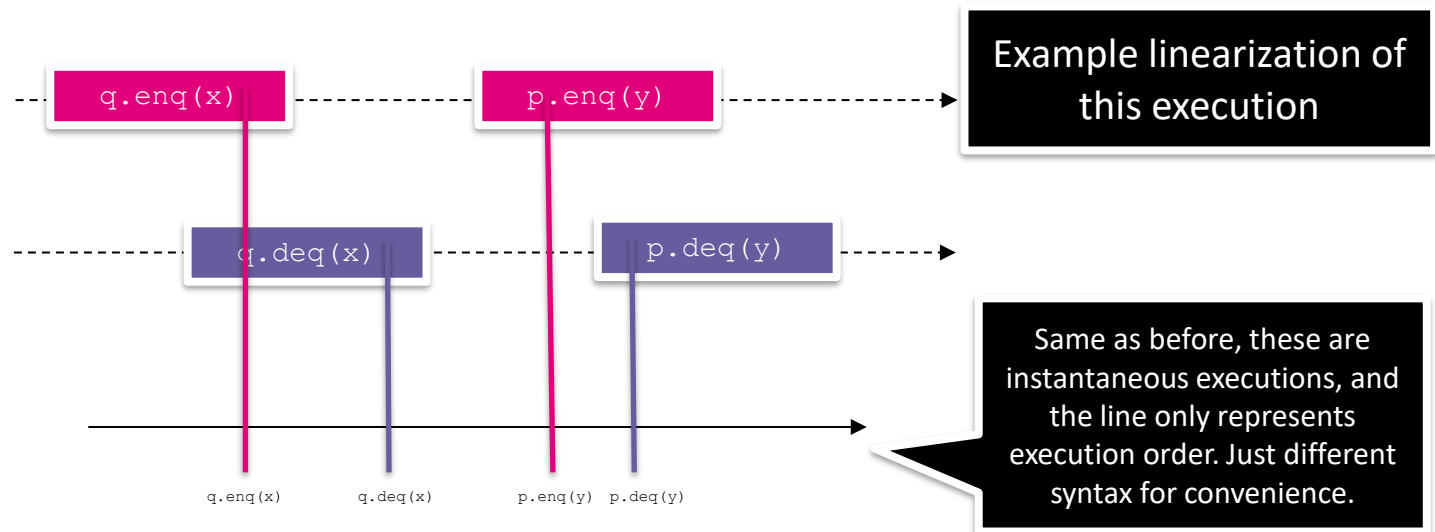


- ***To show that a concurrent execution is linearizable, we must define linearization points within each method call, which project to a sequential execution that satisfies the specification of the object***
 - A *linearization point* defines the instant in the execution when the method call “takes effect”



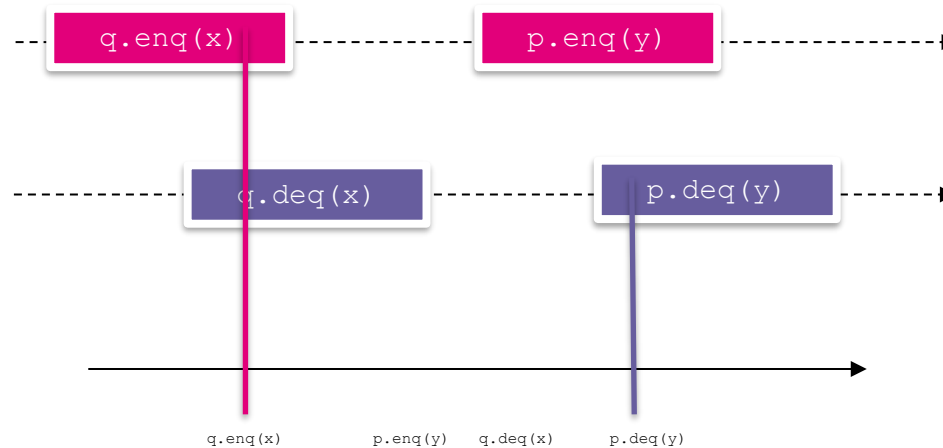


- ***To show that a concurrent execution is linearizable, we must define linearization points within each method call, which project to a sequential execution that satisfies the specification of the object***
 - A *linearization point* defines the instant in the execution when the method call “takes effect”





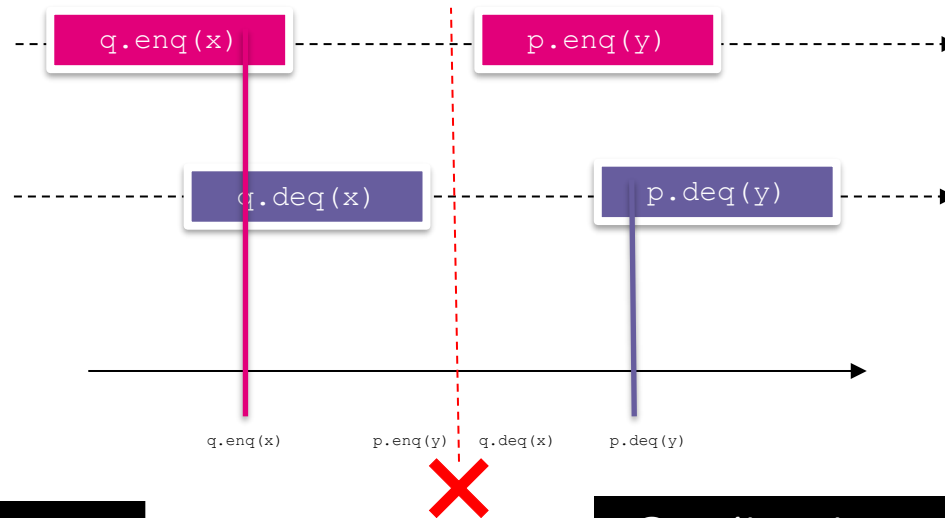
- To show that a concurrent execution is linearizable, we must define linearization points within each method call, which project to a sequential execution that satisfies the specification of the object
 - A *linearization point* defines the instant in the execution when the method call “takes effect”



The sequentially consistent order from before now is not a valid linearization!

The reason is that it does not satisfy the real time ordering

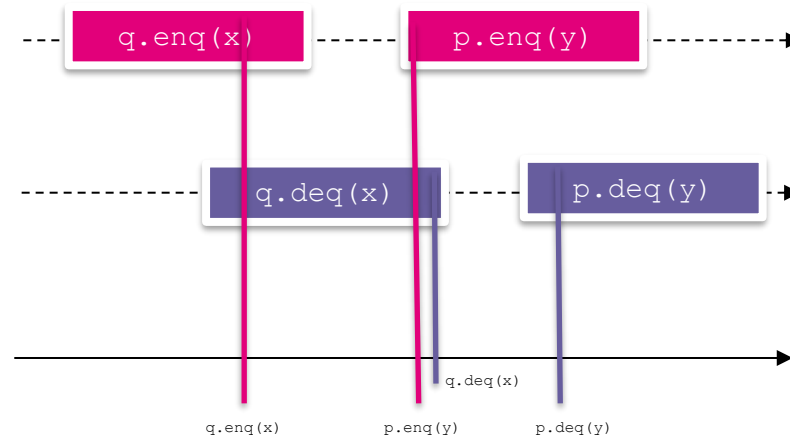
- To show that a concurrent execution is linearizable, we must define linearization points within each method call, which project to a sequential execution that satisfies the specification of the object
 - A *linearization point* defines the instant in the execution when the method call “takes effect”



The sequentially consistent order from before now is not a valid linearization!

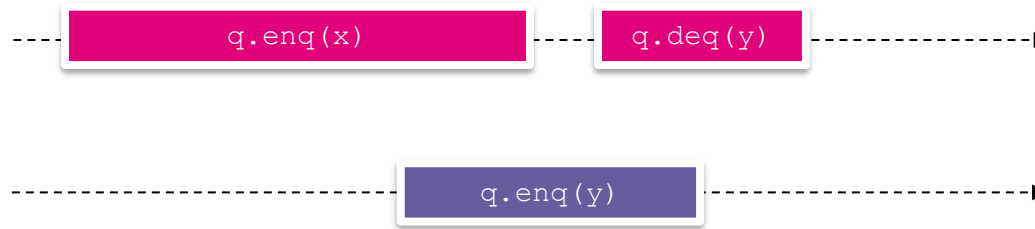
The reason is that it does not satisfy the real time ordering

- To show that a concurrent execution is linearizable, we must define linearization points within each method call, which project to a sequential execution that satisfies the specification of the object
 - A *linearization point* defines the instant in the execution when the method call “takes effect”



If $q.deq(x)$ and $p.enq(y)$ overlap, then the sequentially consistent order is a valid linearization

Sequential consistency



Is this concurrent execution linearizable?
(Recall we are working with a FIFO queue)



menti.com
6658 7456



- Until now, linearizability was presented as a property of *executions*, not concurrent objects
- A concurrent object is linearizable iff all concurrent executions of method calls are linearizable
- **Linearizability is a compositional property**
 - If two objects are linearizable, any concurrent execution involving the two objects is guaranteed to be linearizable
 - Correctness proofs can be split into single objects that later can be safely composed
- To show that an object implementation is linearizable first we must select its linearization points in the source code of the object class
 - Very often linearization points correspond to CAS operations

Proving this is hard

Linearizable Concurrent Objects

Typically, in each method, there is one linearization for each of the outcomes after finishing the execution of the method



· 47

```
class WaitFreeQueue<T> {
    int head = 0, tail = 0;
    T[] items;

    public WaitFreeQueue(int capacity) {
        items = (T[]) new Object[capacity]
    }

    public void enq(T x) throws Exception {
        if (tail - head == items.length) // E1
            thrown new Exception(); // E2
        items[tail % items.length] = x; // E3
        tail++; // E4
    }

    public void deq() throws Exception {
        if (tail - head == 0) // D1
            thrown new Exception(); // D2
        T x = items[head % items.length]; // D3
        head++; // D4
        return x;
    }
}
```

Recall to assume all variables have volatile semantics and each program line is atomic.

Linearization points for enqueue

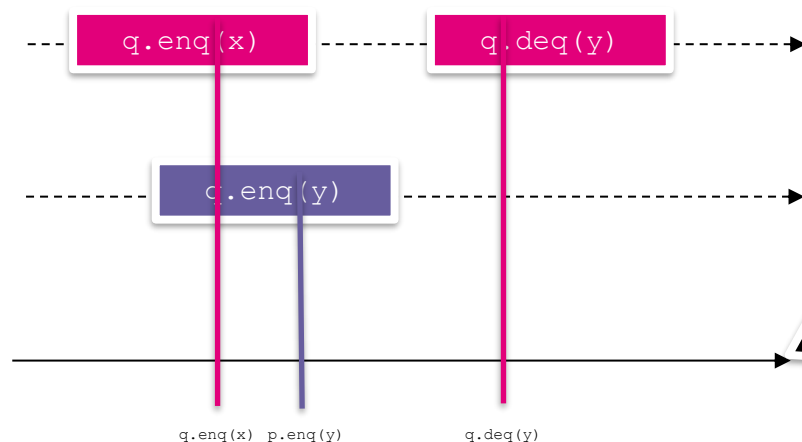
- E4 is the point when an enqueue is completed if the queue is not full
- E1 is the point when an enqueue is completed if the queue is full

Linearization points in dequeue

- D4 is the point when a dequeue is completed if the queue is not empty
- D1 is the point when a dequeue is completed when queue is empty



- Finding linearizable points in the object class is not enough to show correctness
 - Consider this interleaving: E1, E3, E1, E3, E4, E4, D1, D3, D4
 - It corresponds to a linearization of this kind:



The graphical concurrent execution is only for illustrative purposes:

- It represents the textual interleaving above
- We cannot change move the linearization points

This does not satisfy the sequential spec of a FIFO queue!

Thus, we can conclude that WaitFreeQueue is not linearizable

Linearizability of MS Queue

Consult this slide for
exercise 7.2.1

· 49



```
class MSQueue<T> implements UnboundedQueue<T> {  
...  
    public void enqueue(T item) {  
        Node<T> node = new Node<T>(item, null);  
        while (true) {  
            Node<T> last = tail.get();  
            Node<T> next = last.next.get();  
            if (last == tail.get()) { // E7  
                if (next == null) { // E8  
                    // In quiescent state, try inserting new node  
                    if (last.next.compareAndSet(next, node)) { // E9  
                        // Insertion succeeded, try advancing tail  
                        tail.compareAndSet(last, node);  
                        return;  
                    }  
                } else  
                    // Queue in intermediate state, advance tail  
                    tail.compareAndSet(last, next);  
            }  
        }  
    }  
...  
}
```

- Enqueue has one linearization point:
 - E9 – if successfully executed, the element has been enqueued
- Correctness (informal but systematic, tries to cover all branches):
 - If two threads execute enqueue concurrently before tail and next are updated, then only one of them succeeds in executing E9 (and possibly update the tail). The other fails and repeats the enqueueing
 - If a thread executes enqueue after another thread updated the tail, then E7 fails and it repeats the enqueue
 - If a thread executes enqueue after another thread updated next, then E8 fails, the thread tries to advance the tail, and it restarts the enqueue

Linearizability of MS Queue

Consult this slide for
exercise 7.2.1

· 50

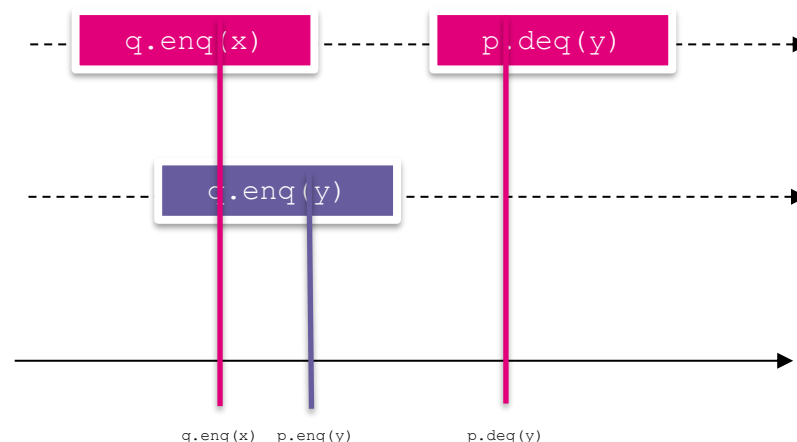


```
class MSQueue<T> implements UnboundedQueue<T> {
...
    public T dequeue() {
        while (true) {
            Node<T> first = head.get();
            Node<T> last = tail.get();
            Node<T> next = first.next.get(); // D3
            if (first == head.get()) { // D5
                if (first == last) { // D6
                    if (next == null) // D7
                        return null;
                    else
                        tail.compareAndSet(last, next);
                } else {
                    T result = next.item;
                    if (head.compareAndSet(first, next)) // D13
                        return result;
                }
            }
        }
    }
...
}
```

- Dequeue has two linearization points
 - D3 - if the queue is empty. After its execution, the evaluation of D7 is determined and the method will return null.
 - D13 - if successfully executed, the element has been dequeued
- Correctness (informal but systematic, tries to cover all branches):
 - If two threads execute dequeue concurrently before the head is updated (D5 succeeds for both) and the queue is not empty (D6 fails), then D13 succeeds for only one of them. The other restarts the dequeue
 - If a thread executes dequeue after another thread updated the head, then D5 fails and it restarts the dequeue
 - If a thread execute dequeue while another thread executed enqueue (E9) and before the enqueueing thread updates the tail, then D7 fails, the dequeuing thread tries to update the tail and restarts the dequeue

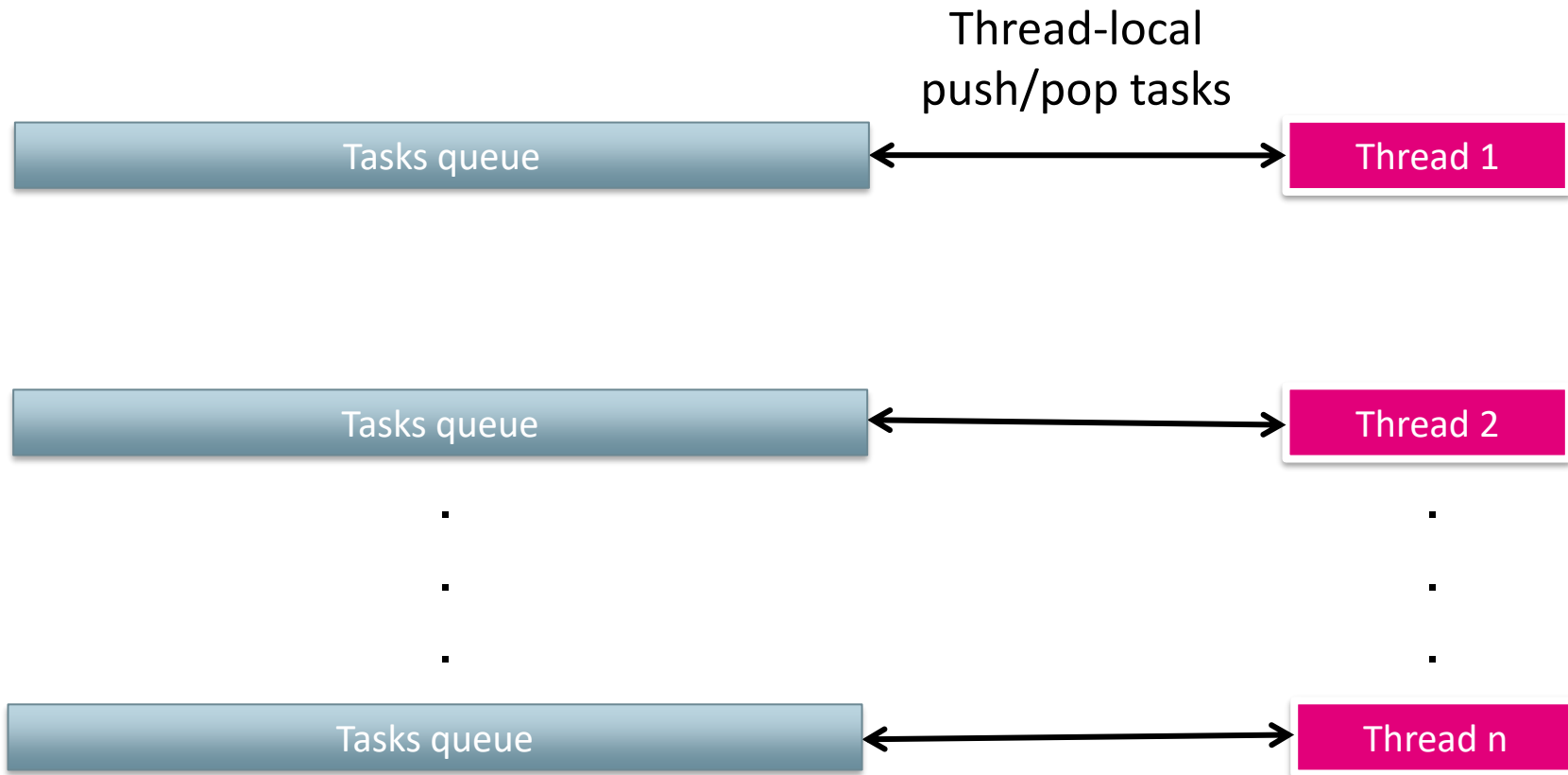


- The correctness arguments in the previous slides increase our confidence that all linearizations satisfy the FIFO queue spec
- This linearization is impossible in the MS Queue
- If pink executes E9 before purple, then either:
 - Purple restart the enqueue due to CAS failure when updating next
 - Purple restart the enqueue because the tail was updated by pink
 - Purple updates the tail due to pink not having done so yet
- Note that in no case y is enqueued before x (or replaced)



Work-stealing queues

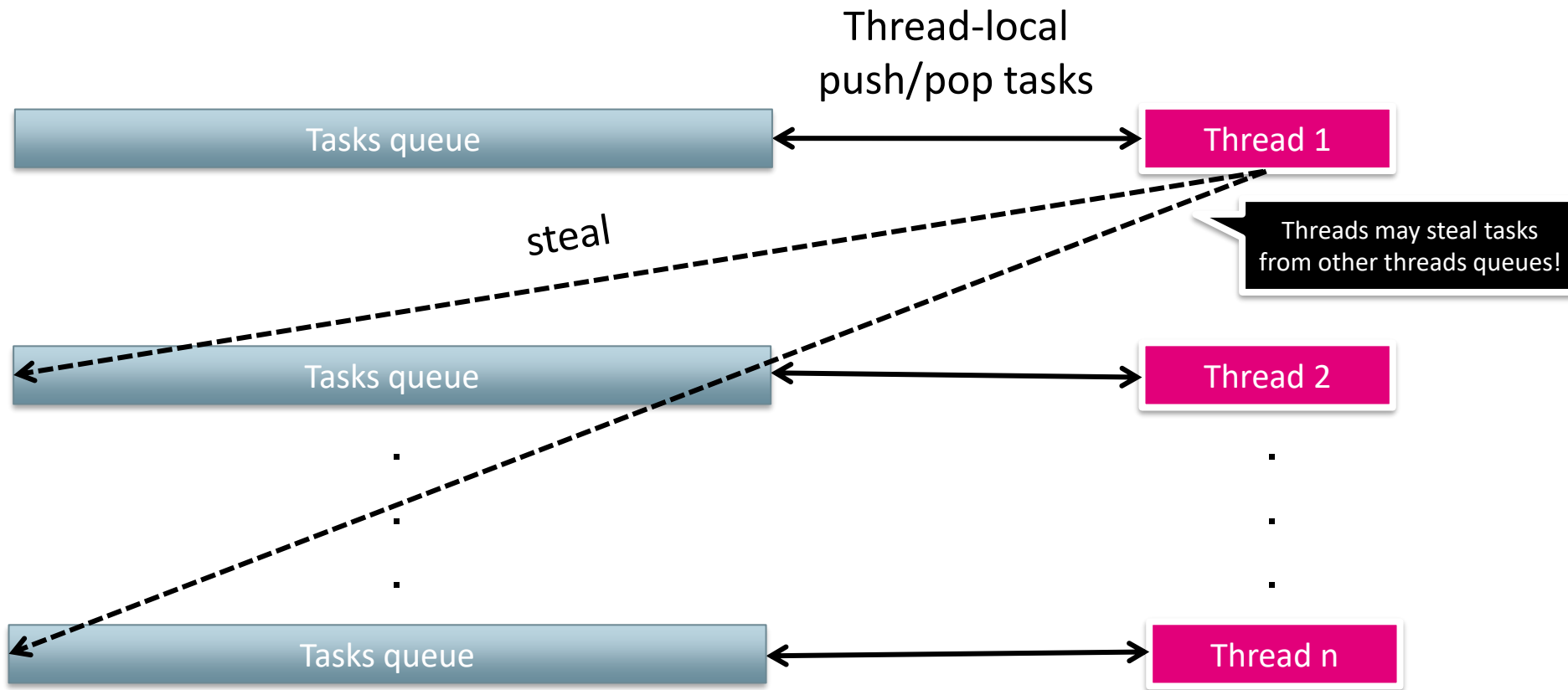
Work-stealing task queue (intuition)



Work-stealing task queue (intuition)



· 56



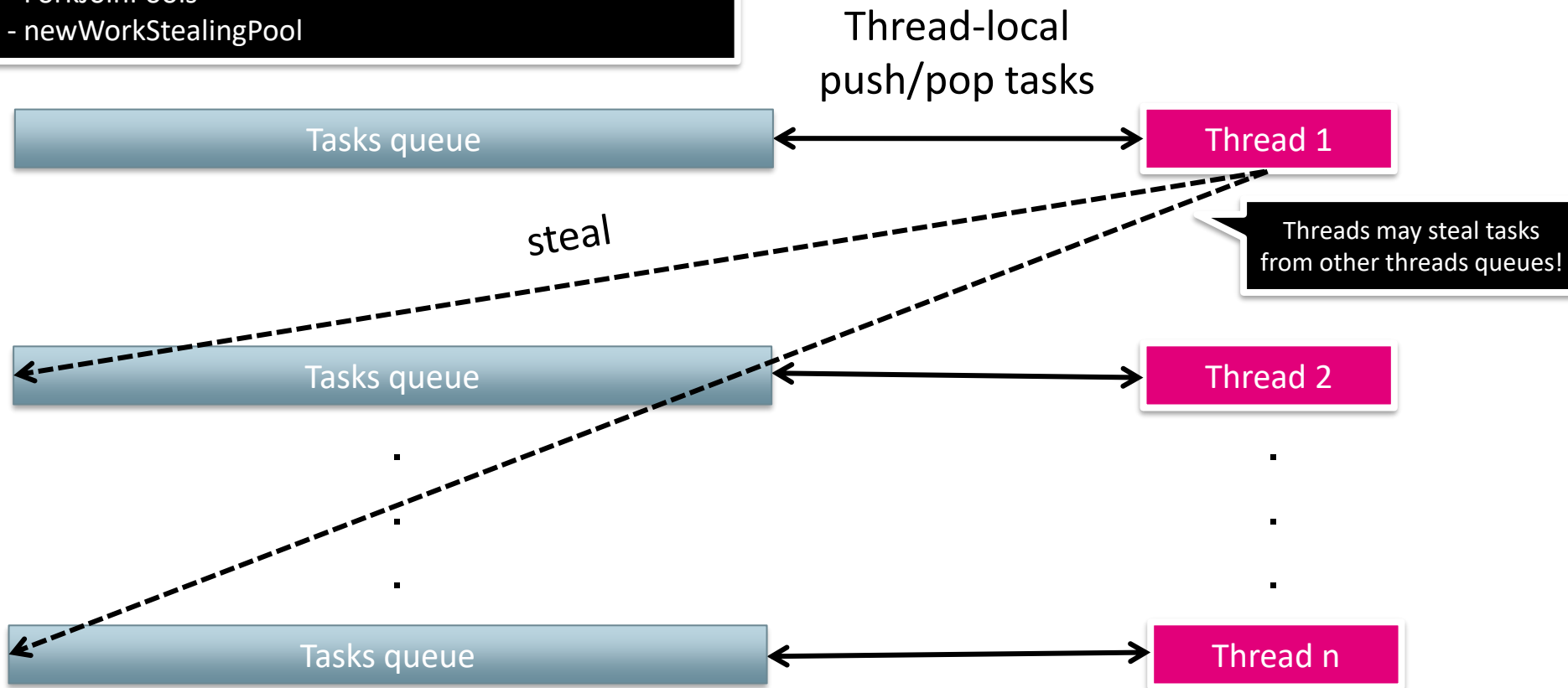
Work-stealing task queue (intuition)



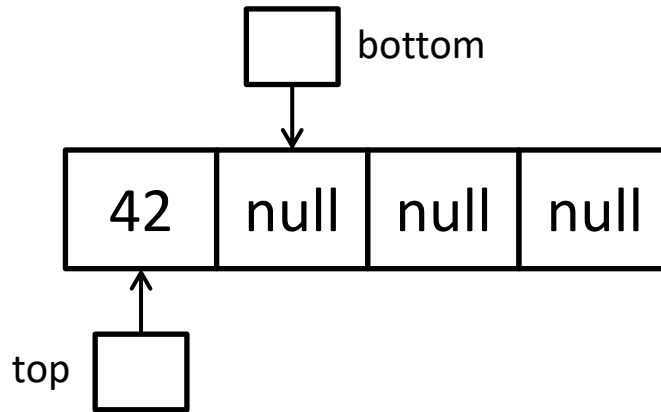
· 57

It is used in the implementation thread pools:

- ForkJoinPools
- newWorkStealingPool



- A work-stealing queue has the following methods
 - Push – adds an element at the bottom of the queue (thread-local)
 - Pop – removes an element from the bottom of the queue (thread-local)
 - Steal – removes an element from the top of the queue (concurrent)



```
interface Deque<T> {  
    void push(T item); // at bottom  
    T pop();           // from bottom  
    T steal();         // from top  
}
```

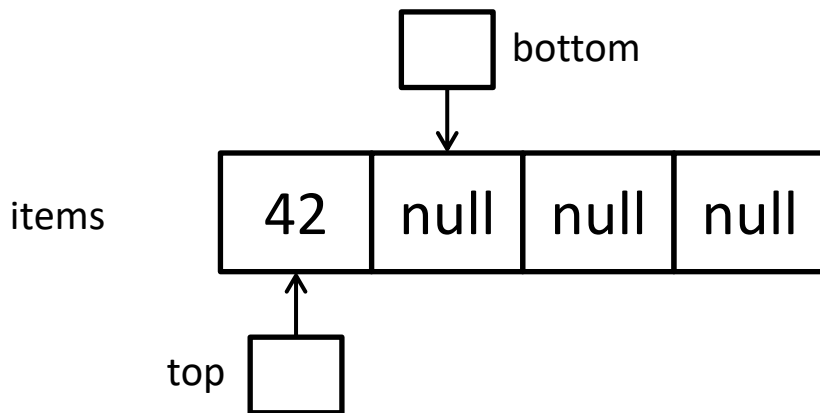
We consider a simplified implementation with a fix size array

Chase-Lev work-stealing queue - state



· 59

```
class ChaseLevDeque<T> implements Deque<T> {  
    private volatile long bottom = 0;  
    private final AtomicLong top = new AtomicLong();  
    private final T[] items;  
    ...  
}
```

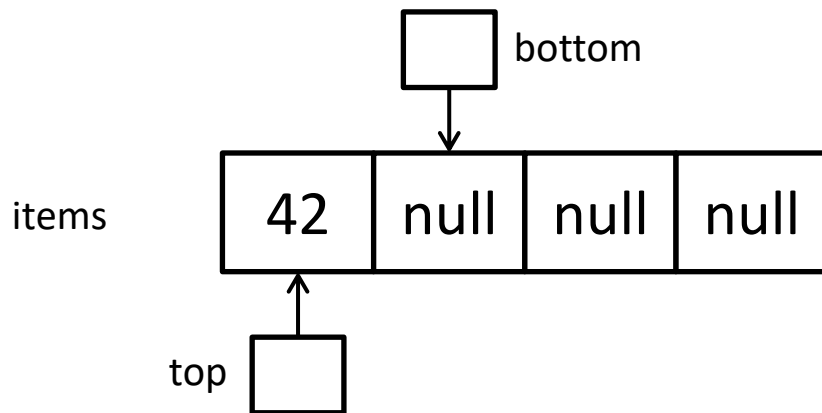


- The variable bottom is thread-local
 - Only the thread assigned to the queue can write it (other threads may read it)
- Any thread can read/write the variable top
 - We need an atomic variable to prevent data races
- For simplicity, we consider a fix-size array to store the elements of the queue
 - The array is used as a circular buffer

Chase-Lev work-stealing queue - state



```
class ChaseLevDeque<T> implements Deque<T> {  
    private volatile long bottom = 0;  
    private final AtomicLong top = new AtomicLong();  
    private final T[] items;  
    ...  
}
```



- The variable `bottom` is thread-local
 - Only the thread assigned to the queue can write it (other threads may read it); thus `volatile` is sufficient
- Any thread can read/write the variable `top`
 - We need an atomic variable to prevent data races
- For simplicity, we consider a fix-size array to store the elements of the queue
 - The array is used as a circular buffer

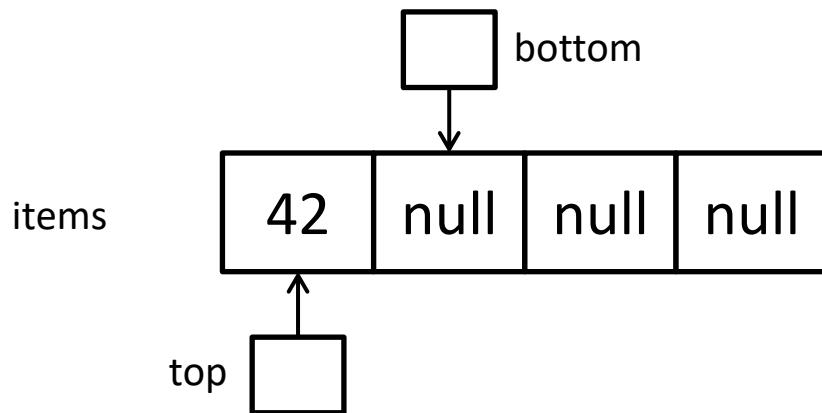
Chase-Lev work-stealing queue - push



· 61

```
public void push(T item) { // at bottom
    final long b = bottom, t = top.get(), size = b - t;
    if (size == items.length)
        throw new RuntimeException("queue overflow");
    items[index(b, items.length)] = item;
    bottom = b+1;
}
```

- It is race condition free because it is assumed to be thread-local
 - Always the same thread executes this method
 - The thread only writes in bottom



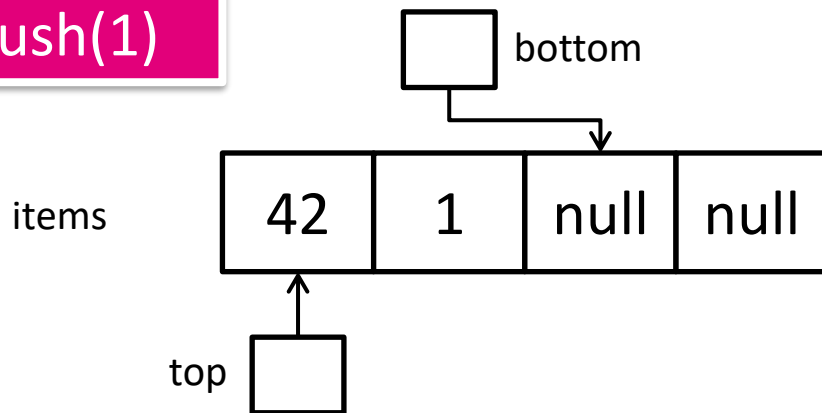
Chase-Lev work-stealing queue - push



· 62

```
public void push(T item) { // at bottom
    final long b = bottom, t = top.get(), size = b - t;
    if (size == items.length)
        throw new RuntimeException("queue overflow");
    items[index(b, items.length)] = item;
    bottom = b+1;
}
```

push(1)



- It is race condition free because it is assumed to be thread-local
 - Always the same thread executes this method
 - The thread only writes in bottom

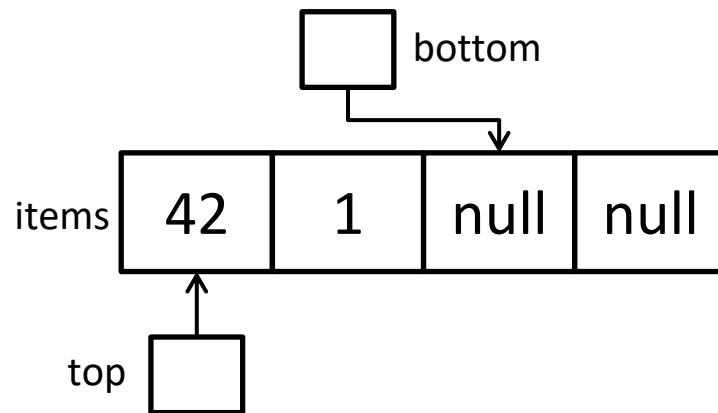
Chase-Lev work-stealing queue - steal



· 63

```
public T steal() { // from top
    final long t = top.get();
    final long b = bottom;
    final long size = b - t;
    if (size <= 0)
        return null;
    else {
        T result = items[index(t, items.length)];
        if (top.compareAndSet(t, t+1))
            return result;
        else
            return null;
    }
}
```

- It is executed by multiple threads
- Only reads bottom
- Performs a CAS on top to steal the top element
 - Only if not empty



Chase-Lev work-stealing queue - steal

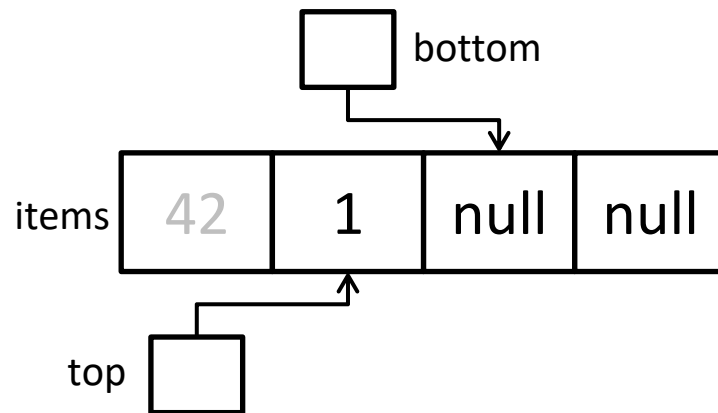


· 64

```
public T steal() { // from top
    final long t = top.get();
    final long b = bottom;
    final long size = b - t;
    if (size <= 0)
        return null;
    else {
        T result = items[index(t, items.length)];
        if (top.compareAndSet(t, t+1))
            return result;
        else
            return null;
    }
}
```

steal() -> 42

- It is executed by multiple threads
- Only reads bottom
- Performs a CAS on top to steal the top element
 - Only if not empty



Chase-Lev work-stealing queue - steal



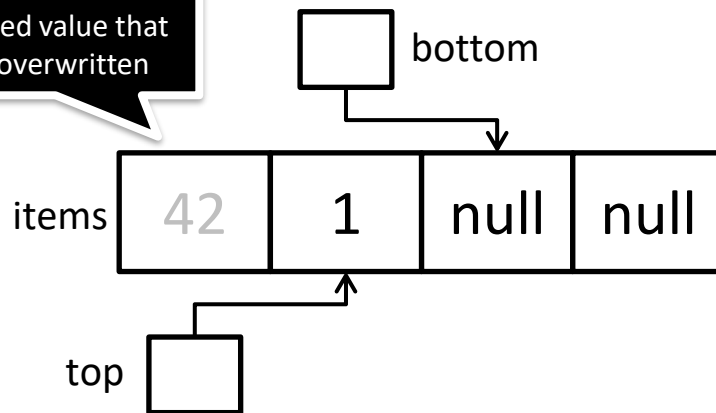
· 65

```
public T steal() { // from top
    final long t = top.get();
    final long b = bottom;
    final long size = b - t;
    if (size <= 0)
        return null;
    else {
        T result = items[index(t, items.length)];
        if (top.compareAndSet(t, t+1))
            return result;
        else
            return null;
    }
}
```

steal() -> 42

- It is executed by multiple threads
- Only reads bottom
- Performs a CAS on top to steal the top element
 - Only if not empty

This becomes a deprecated value that will be overwritten



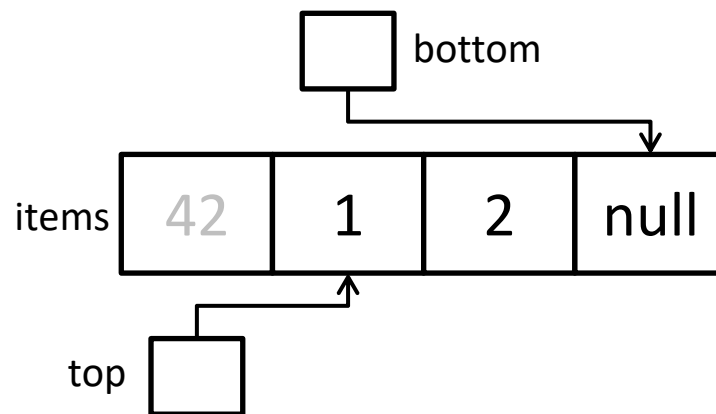
Chase-Lev work-stealing queue - pop

· 66



```
public T pop() { // from bottom
    final long b = bottom - 1;
    bottom = b;
    final long t = top.get(),
    final long afterSize = b - t;
    if (afterSize < 0) {
        bottom = t;
        return null;
    } else {
        T result = items[index(b, items.length)];
        if (afterSize > 0)
            return result;
        else {
            if (!top.compareAndSet(t, t+1))
                result = null;
            bottom = t+1;
            return result;
        }
    }
}
```

- Thread-local but more subtle than push
- It updates bottom (thread-local) and possibly top (concurrent)



Chase-Lev work-stealing queue - pop

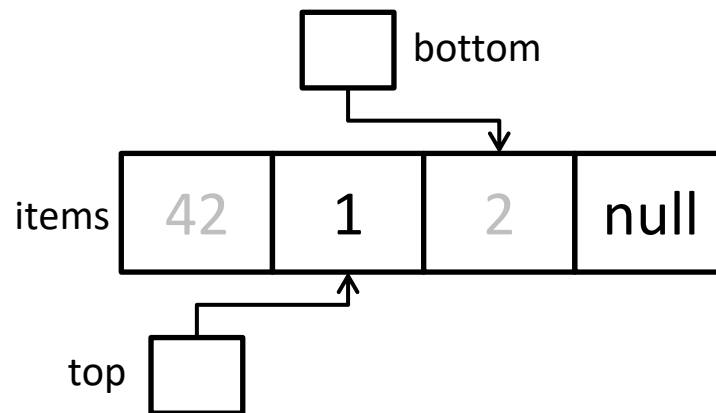
· 67



```
public T pop() { // from bottom
    final long b = bottom - 1;
    bottom = b;
    final long t = top.get(),
    final long afterSize = b - t;
    if (afterSize < 0) {
        bottom = t;
        return null;
    } else {
        T result = items[index(b, items.length)];
        if (afterSize > 0)
            return result;
        else {
            if (!top.compareAndSet(t, t+1))
                result = null;
            bottom = t+1;
            return result;
        }
    }
}
```

pop() -> 2

- When only the assign thread executes, then we simply update bottom and return the element

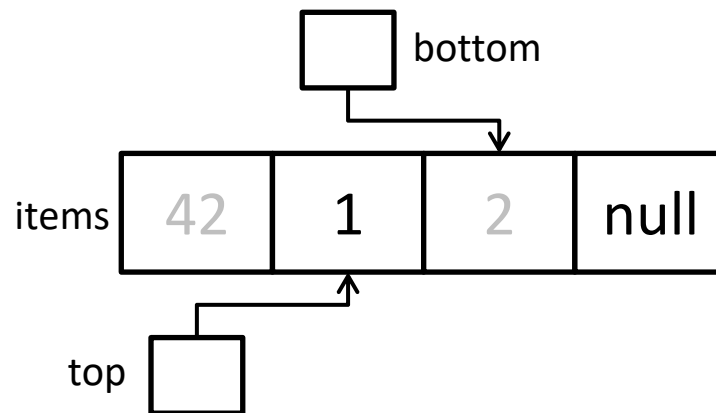


Chase-Lev work-stealing queue - pop



```
public T pop() { // from bottom
    final long b = bottom - 1;
    bottom = b;
    final long t = top.get(),
    final long afterSize = b - t;
    if (afterSize < 0) {
        bottom = t;
        return null;
    } else {
        T result = items[index(b, items.length)];
        if (afterSize > 0)
            return result;
        else {
            if (!top.compareAndSet(t, t+1))
                result = null;
            bottom = t+1;
            return result;
        }
    }
}
```

- What if we had pop() and steal() concurrently?



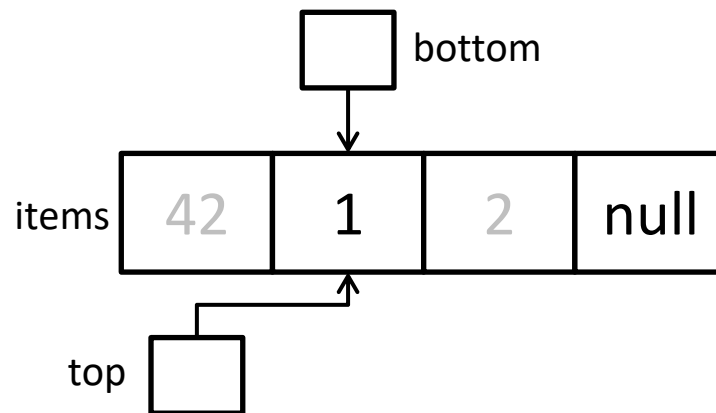
Chase-Lev work-stealing queue - pop



```
public T pop() { // from bottom
    final long b = bottom - 1;
    bottom = b;
    final long t = top.get(),
    final long afterSize = b - t;
    if (afterSize < 0) {
        bottom = t;
        return null;
    } else {
        T result = items[index(b, items.length)];
        if (afterSize > 0)
            return result;
        else {
            if (!top.compareAndSet(t, t+1))
                result = null;
            bottom = t+1;
            return result;
        }
    }
}
```

pop() -> ?

- What if we had pop() and steal() concurrently?



```

public T pop() { /
    final long b = bottom;
    bottom = b;
    final long t = top.get();
    final long afterSize = b - t;
    if (afterSize < 0) {
        bottom = t;
        return null;
    } else {
        T result = items[index(b, items.length)];
        if (afterSize > 0)
            return result;
        else {
            if (!top.compareAndSet(t, t+1))
                result = null;
            bottom = t+1;
            return result;
        }
    }
}

```

```

public T steal() { // from top
    final long t = top.get();
    final long b = bottom;
    final long size = b - t;
    if (size <= 0)
        return null;
    else {
        T result = items[index(t, items.length)];
        if (top.compareAndSet(t, t+1))
            return result;
        else
            return null;
    }
}

```

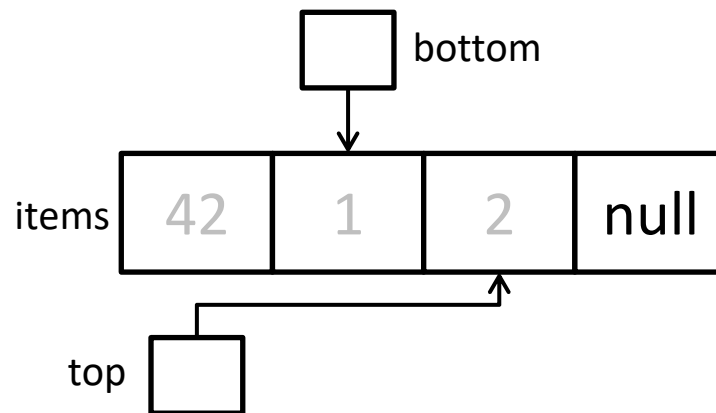
pop() -> ?

steal() -> ?



What if we had pop() and steal() concurrently?

- Whatever thread succeeds in the CAS operation gets the element



```

public T pop() { /
    final long b = bottom;
    bottom = b;
    final long t = top.get();
    final long afterSize = b - t;
    if (afterSize < 0) {
        bottom = t;
        return null;
    } else {
        T result = items[index(b, items.length)];
        if (afterSize > 0)
            return result;
        else {
            if (!top.compareAndSet(t, t+1))
                result = null;
            bottom = t+1;
            return result;
        }
    }
}

```

```

public T steal() { // from top
    final long t = top.get();
    final long b = bottom;
    final long size = b - t;
    if (size <= 0)
        return null;
    else {
        T result = items[index(t, items.length)];
        if (top.compareAndSet(t, t+1))
            return result;
        else
            return null;
    }
}

```

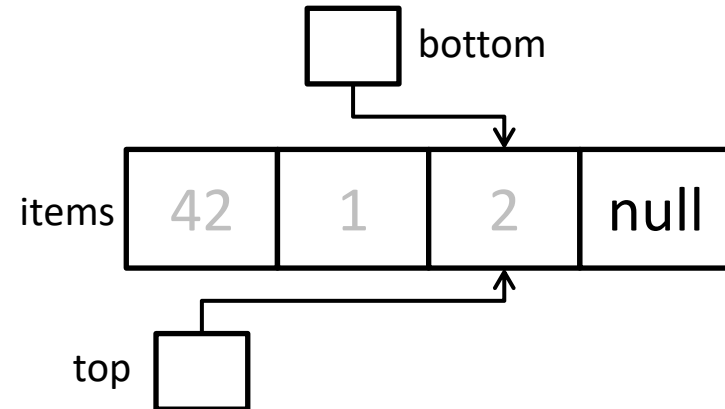
pop() -> ?

steal() -> ?



What if we had pop() and steal() concurrently?

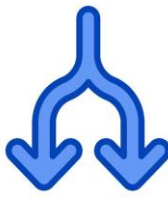
- Whatever thread succeeds in the CAS operation gets the element
- Afterwards, pop always fixes the bottom variable



Prof. Peter Sestoft will teach the following 4 weeks concurrency topics related to performance and scalability



Agenda



- Revisit
 - Progress conditions
 - Lock-free queue
- Linearizability
 - Sequential consistency
 - Definition of Linearizability
 - Linearizable concurrent objects
- Work-stealing queues

Have a nice and well-deserved autumn break!

